Workgroup:	RESTful Provisioning Protocol
Internet-Draft:	draft-ietf-rpp-json-02 (editorial)
Published:	2 November 2026
Intended Status:	Standards Track
Expires:	6 May 2027
Authors:	M. Wullink	P. Kowalik
---	---	---
	SIDN Labs	DENIC

JSON for RESTful Provisioning Protocol (RPP)

Abstract

This document defines the rules for representing the RESTful Provisioning Protocol (RPP) data objects using the JavaScript Object Notation (JSON) Data Interchange Format. It specifies how RPP primitive types, common data types, Component Objects, Data Objects, and associations are mapped to JSON and JSON Schema, and provides normative JSON Schema definitions and examples for the RPP Data Objects, the representation of update requests, and an extension framework.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 6 May 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction 9
1.1. Motivation 9
2. Terminology 9
3. Conventions Used in This Document 10
4. JSON Representation Rules 10
4.1. Primitive Type Mappings 10
4.2. Undefined Values 11
4.3. Cardinality Rules 12
4.4. Mutability Rules 13
4.5. Association Rules 14
4.5.1. Labelled Associations 14
4.5.2. Aggregation 14
4.5.3. Composition 15
4.5.4. Labelled Aggregation 15
4.5.5. Dictionary Aggregation 16
4.5.6. Labelled Composition 17
4.5.7. Dictionary Composition 17
4.6. Object Identifier Rules 18
4.7. External Type Embedding Rules 18
4.8. Process Data Embedding Rules 18
4.9. JSON Schema Definition Rules 19
4.9.1. Undeclared Properties 19
4.9.2. Type Property 20
4.9.3. Authorisation Information in Requests 20
4.10. JSON Schema Composition Rules 20
4.10.1. Status Objects 22
4.10.2. Inheritance 22
4.10.3. RPP Profiles and Validation 23

5. Update Rules 23
5.1. Full 23
5.2. Partial 24
5.2.1. Matching 24
5.2.2. Processing Steps 26
5.3. Schema 30
5.3.1. Examples 31
6. JSON Schema Definitions 33
6.1. Common Data Types Schemas 34
6.1.1. Identifier 34
6.1.2. Client Identifier 34
6.1.3. Phone Number 34
6.2. Component Objects Schemas 35
6.2.1. Period Object 35
6.2.2. Provisioning Metadata Object 36
6.2.3. Status Objects 36
6.2.3.1. Domain Name Status Object 37
6.2.3.2. Host Status Object 39
6.2.3.3. Contact Status Object 40
6.2.3.4. Organisation Role Status Object 42
6.2.3.5. Organisation Status Object 42
6.2.3.6. User Status Object 44
6.2.4. DNS Resource Record Object 45
6.2.5. DNS Operational Controls Object 46
6.2.6. DNS Data Object 47
6.2.7. Authorisation Information Object 49
6.2.8. Disclose Object 49
6.2.9. JSContact Card Object 50
6.2.10. Restore Report Object 52
6.2.11. Organisation Role Object 53

6.2.12. Extension Object 54
6.2.13. Discovery 54
6.2.13.1. Version Object 54
6.2.13.2. Environment Object 55
6.2.13.3. IDN LGR Object 55
6.2.13.4. Notice Object 56
6.2.14. Processes Object 56
6.2.15. Availability 57
6.2.15.1. Base Availability Object 57
6.2.15.2. Domain Name Availability Object 58
6.2.15.3. Contact Availability Object 58
6.2.15.4. Host Availability Object 59
6.2.16. Messages 59
6.2.16.1. Base Message Object 59
6.2.16.2. Review Response Object 60
6.2.16.3. Transfer Request Message Object 61
6.2.16.4. Transfer Outcome Message Object 62
6.2.16.5. Expiration Deletion Message Object 63
6.2.16.6. Auto-Renewal Message Object 64
6.2.16.7. Service Notice Message Object 64
6.3. Process Object Schemas 65
6.3.1. Base Process Object 65
6.3.2. Create Process Object 65
6.3.3. Renew Process Object 66
6.3.4. Transfer Process Object 68
6.3.4.1. Create 68
6.3.4.2. Read 69
6.3.4.3. Approve 71
6.3.4.4. Cancel 71
6.3.4.5. Reject 71

6.3.4.6. Reference 71
6.3.5. Restore Process Object 72
6.4. Domain Name Data Object 74
6.4.1. Availability 74
6.4.2. Create 74
6.4.3. Create Process 75
6.4.4. Read 77
6.4.5. Update 78
6.4.6. Transfer 79
6.4.7. Reference 82
6.5. Contact Data Object 83
6.5.1. JSContact Profile for RPP 83
6.5.2. Availability 83
6.5.3. Create 83
6.5.4. Read 84
6.5.5. Update 84
6.5.6. Reference 85
6.6. Host Data Object 85
6.6.1. Availability 85
6.6.2. Create 86
6.6.3. Read 86
6.6.4. Update 87
6.6.5. Reference 87
6.7. Organisation Data Object 87
6.7.1. Create 88
6.7.2. Read 89
6.7.3. Update 90
6.7.4. Reference 91
6.8. Message Data Object 92
6.8.1. Create 92

6.8.2. Read 93
6.8.3. Query 94
6.8.4. Reference 94
6.9. User Data Object 95
6.9.1. Create 95
6.9.2. Read 95
6.9.3. Update 96
6.9.4. Reference 96
6.10. Discovery Data Object 97
6.10.1. Read 97
6.11. Profile Data Object 99
6.11.1. Read 99
6.11.2. Reference 100
7. Extension Framework 101
7.1. Extension Mapping 102
7.2. Effective Schema 102
8. Examples 104
8.1. Domain Name 104
8.1.1. Availability 104
8.1.2. Create 105
8.1.3. Create Process Data 106
8.1.4. Read 106
8.1.5. Update 108
8.1.6. Delete 109
8.1.7. Renew 109
8.1.8. Transfer Request 109
8.1.9. Transfer Query 110
8.1.10. Transfer Approve 110
8.1.11. Transfer Cancel 111
8.1.12. Transfer Reject 111

8.1.13. Restore Request 111
8.1.14. Restore Report 112
8.1.15. Read (Restore Query) 113
8.2. Contact 113
8.2.1. Availability 113
8.2.2. Create 113
8.2.3. Read 115
8.2.4. Update 116
8.2.5. Delete 118
8.2.6. Transfer Request 118
8.2.7. Transfer Query 119
8.2.8. Transfer Cancel / Approve 119
8.2.9. Transfer Reject 119
8.3. Host 120
8.3.1. Availability 120
8.3.2. Create 120
8.3.3. Read 121
8.3.4. Update 122
8.3.5. Delete 123
8.3.6. Restore Request 123
8.3.7. Restore Report 124
8.3.8. Read (Restore Query) 125
8.4. Organisation 125
8.4.1. Create 125
8.4.2. Read 127
8.4.3. Update 128
8.4.4. Delete 129
8.4.5. Reference 129
8.5. User 129
8.5.1. Create 129

8.5.2. Read 130
8.5.3. Update 131
8.5.4. Delete 131
8.5.5. Reference 131
8.6. Message 131
8.6.1. Create 131
8.6.2. Read 132
8.6.3. Query 135
8.7. Discovery 136
8.7.1. Read 136
8.8. Profile 138
8.8.1. Read 138
9. IANA Considerations 138
9.1. RPP Media Type (application/rpp+json) 138
9.2. RPP JSON Schema Extensions Registry 139
10. Internationalization Considerations 140
11. Security Considerations 140
12. Acknowledgments 141
13. Change History 141
13.1. Version draft-ietf-rpp-json-01 to draft-ietf-rpp-json-02 141
13.2. Version draft-ietf-rpp-json-00 to draft-ietf-rpp-json-01 141
13.3. Version draft-wullink-rpp-json-02 to draft-ietf-rpp-json-00 141
13.4. Version 01 to 02 142
13.5. Version 00 to 01 142
14. References 142
14.1. Normative References 142
14.2. Informative References 144
Authors' Addresses 144

1. Introduction

The RESTful Provisioning Protocol (RPP) defines a set of data objects for managing registry resources, such as domain names, contacts, hosts, organisations, and users. The data model is defined in [I-D.ietf-rpp-data-objects] independently of any particular representation format. This document defines the JSON [RFC8259] representation of those data objects.

JSON has emerged as the de facto standard data format for modern RESTful APIs. Its widespread adoption across tools, libraries, and developer communities makes it well suited as the primary representation format for RPP. This document provides the normative rules and JSON Schema definitions required for implementations to produce and consume RPP messages in JSON.

The separation between the abstract data model and its concrete JSON representation ensures that the protocol's semantic foundation remains stable while enabling the adoption of JSON across diverse deployment environments.

1.1. Motivation

The RESTful Provisioning Protocol (RPP) introduces a new provisioning mechanism that aligns more closely with modern cloud infrastructure, enhancing the scalability of server deployments. While RESTful protocols do not mandate a specific media type for resource description, the widespread adoption of JSON in web services has established it as the de facto standard for modern APIs. The increasing availability of tools, software libraries, and a skilled workforce has led several registries to adopt JSON for data exchange within their API ecosystems. Registries supporting JSON can offer a unified API ecosystem that extends beyond domain name and IP address provisioning, maintaining a consistent technology stack, data formats, and developer experience.

JSON's syntax, known for its straightforwardness and minimal verbosity, significantly eases the tasks of writing, reading, and maintaining code. This simplicity is especially advantageous for the rapid comprehension and integration of provisioning APIs.

The lightweight nature of JSON can result in faster processing and data transfers, a critical aspect in high-volume transaction environments such as domain registration. Enhanced API response times can lead to more efficient domain lookups, registrations, and updates. JSON parsing is typically fast and well-supported by standard libraries, contributing to improved system performance amid frequent interactions between RPP clients and servers.

However, the absence of a standardised JSON format for domain provisioning has led to the emergence of TLD-specific implementations that lack interoperability, increasing the development effort required for integration. Similarly, at the registrar level, the absence of standards has resulted in numerous incompatible API implementations provided to clients and resellers. Standardising a JSON format for domain provisioning within the RPP framework could mitigate these challenges, reducing fragmentation and simplifying integration efforts across the domain registration industry.

2. Terminology

In this document, the following terminology is used.

Terms of the RPP data model, such as Data Object, Component Object, Process Object, Common Data Type, data element, cardinality, mutability, Identifier, Unique Identifier, Input Data Element, Parameter, Visibility, Direct Access, Input, Output, Reference, Owner Data Object, and This Object, are used as defined in [I-D.ietf-rpp-data-objects].

RPP Data Objects - The abstract data model definitions of the objects provisioned and exchanged with RPP, as specified in [I-D.ietf-rpp-data-objects].

RESTful Provisioning Protocol - A RESTful protocol for provisioning heterogeneous database objects.

JSON Schema - A vocabulary that allows annotation and validation of JSON documents, as described in [I-D.ietf-jsonschema-json-schema].

EPP Compatibility Profile - A set of additional constraints defined in [I-D.ietf-rpp-data-objects] that a server MUST adhere to when supporting both RPP and EPP concurrently.

3. Conventions Used in This Document

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC2119].

JSON is case sensitive. Unless stated otherwise, JSON specifications and examples provided in this document MUST be interpreted in the character case presented. The examples in this document assume that request and response messages are properly formatted JSON documents. Indentation and white space in examples are provided only to illustrate element relationships and for improving readability, and are not REQUIRED features of the protocol.

All JSON Schema definitions in this document use JSON Schema draft 2020-12 [I-D.ietf-jsonschema-json-schema], and where not provided with a `$schema` keyword, the following default applies:

```
"$schema": "https://json-schema.org/draft/2020-12/schema"
```

4. JSON Representation Rules

This section defines the normative rules for representing the RPP data model in JSON. The data model defines all primitive types, common data types, component objects, resource objects, and associations independently of any concrete representation format. The rules in this section specify how those abstract definitions map to JSON and JSON Schema version 2020-12.

4.1. Primitive Type Mappings

RPP primitive types MUST be represented in JSON as follows:

RPP Primitive Type	JSON Type	Notes
String	`string`	Unicode character sequence
Integer	`number`	Whole number, positive or negative
Boolean	`boolean`	`true` or `false`
Decimal	`string`	Base-10 fractional value with exact representation within a defined precision. Number is encoded into string same as `"number"` in [RFC8259] without exponent part `"ext"`.
Date	`string`	Full-date as per [RFC3339], e.g. `"2025-10-27"`
Timestamp	`string`	Date-time in UTC as per [RFC3339], e.g. `"2025-10-27T09:42:51Z"`
URL	`string`	Uniform Resource Locator as per [RFC1738]
Binary	`string`	Base64-encoded binary data

Table 1

4.2. Undefined Values

A data element can have no defined value, for example when the value is NULL in the server's database or has never been set. The JSON `null` value must not be used to represent an undefined value, a JSON Schema type such as `"type": "string"` does not allow `null`.

An undefined value of a primitive type data element must not be rendered, the property must be omitted from the JSON object. An array data element with cardinality `0+` that has no values may be rendered as an empty array (`[]`) or may be omitted from the JSON object. Consumers must treat both representations as equivalent.

Example: a domain name for which the primitive type data element `expiryDate` is undefined and not rendered, the array data element `nameservers` has no values, with `nameservers` rendered as an empty array:

```
{
    "@type": "domainName",
    "name": "example.example",
    "provMetadata": {
        "@type": "provMetadata",
        "spClientId": "ClientX"
    },
    "nameservers": []
}
```

The same domain name with the `nameservers` array omitted is equivalent:

```
{
    "@type": "domainName",
    "name": "example.example",
    "provMetadata": {
        "@type": "provMetadata",
        "spClientId": "ClientX"
    }
}
```

4.3. Cardinality Rules

The cardinality of each data element in the RPP data model MUST be represented as follows in JSON:

A data element with cardinality `1` (exactly one) MUST be represented as a JSON property and MUST be present in the containing JSON object. The element MUST be listed under `required` in the corresponding JSON Schema.

```
{
  "type": "object",
  "properties": {
    "name": { "type": "string" }
  },
  "required": ["name"]
}
```

A data element with cardinality `0-1` (zero or one) MUST be represented as an optional JSON property. The element MUST NOT be listed under `required` in the corresponding JSON Schema. When absent, the element MUST be omitted from the JSON object (not represented as `null`).

```
{
  "type": "object",
  "properties": {
    "expiryDate": { "type": "string", "format": "date-time" }
  }
}
```

A data element with cardinality `0+` (zero or more) MUST be represented as an optional JSON array. When no values are present, the property MUST be omitted or represented as an empty array.

```
{
  "type": "object",
  "properties": {
    "status": {
      "type": "array",
      "items": { "$ref": "#/$defs/domainStatus.read" }
    }
  }
}
```

A data element with cardinality `1+` (one or more) MUST be represented as a required JSON array with `"minItems": 1` and the element MUST be listed under `required` in the corresponding JSON Schema.

```
{
  "type": "object",
  "properties": {
    "records": {
      "type": "array",
      "items": { "$ref": "#/$defs/dnsRecord.read" },
      "minItems": 1
    }
  },
  "required": ["records"]
}
```

4.4. Mutability Rules

Data elements in the RPP data model carry a mutability attribute: `create-only`, `read-write`, or `read-only`. Which of them an object representation contains follows from the Input and Output of the operation:

- Response content contains data elements of every mutability. A data element is left out only when it is not present on the object, when its Visibility is `onDemand` and the client did not request it, or when authorisation or server policy withholds it.
- Request content for an object being created contains its `create-only` and `read-write` data elements. For a Component Object or an Aggregation object creation can take place in create or update operation on the containing object, when a value is set or added.
- Request content for an existing object being modified, i.e. when both the Reference and the Object of the Input are This Object, contains its `read-write` data elements.

Input Data Elements and Parameters are not data elements of the object, have no mutability, and are not subject to the following rules.

Data elements with mutability `read-only` MUST be annotated with `"readOnly": true` in the JSON Schema and MUST NOT be part of any request view.

```
{
  "repositoryId": {
    "type": "string",
    "readOnly": true
  }
}
```

Data elements with mutability `create-only` MUST be annotated with `"writeOnly": true` in request schemas and MUST NOT be part of the `update` view.

Data elements with mutability `read-write` have no additional annotation.

A server MUST reject a request whose content contains a data element that is not allowed in that request, such as a `read-only` data element or a `create-only` data element of an existing object. With the enrichment required by Undeclared Properties, such a request fails JSON Schema validation.

4.5. Association Rules

The RPP data model defines several association types between objects, the following rules specify their JSON representations. An Aggregation represents a relationship between two independent objects, where one object references another. A Composition represents a parent-child relationship where the child object is embedded within the parent object and cannot exist independently.

4.5.1. Labelled Associations

Some associations between objects carry a string label that provides additional context for the relationship. The label is not an identifier of the target object, but rather a descriptor of the association itself. Labelled associations can occur in both aggregations and compositions. When representing labelled associations in JSON, the property `label` MUST be included alongside the reference to the target object. A property with the name `object` MUST be used to contain the reference to the target object, which can be either a limited representation containing at minimum the Unique Identifier of the target object for aggregations or an embedded object for compositions.

4.5.2. Aggregation

An `Aggregation[Type]` represents a relationship between two independent objects. The referenced object is identified by its Unique Identifier. When the cardinality allows more than one target, it MUST be represented as a JSON array. Each element of the array is a limited representation of the referenced object, as specified below.

`Aggregation[Type]` with cardinality `0+` or `1+` MUST be represented as a JSON array of embedded objects. Each object in the array MUST include the data elements of the referenced object type that are relevant to the context (at minimum the Unique Identifier). Other data elements of the referenced object type MAY be included as needed to provide additional context for the client, but are not required. The JSON Schema MUST allow for the presence of these additional fields.

Example: domain nameservers (Aggregation[Host Data Object]) in a read response, returning a limited object representation containing only the Unique Identifier `hostName`:

```
{
    "@type": "domainName",
    "name": "name.example",
    "nameservers": [
        { "@type": "host", "hostName": "ns1.name.example" },
        { "@type": "host", "hostName": "ns2.name.example" }
    ]
}
```

4.5.3. Composition

A `Type` represents a parent-child relationship where the child's lifecycle is bound to the parent and the child cannot exist independently of the parent. In JSON, the child object MUST be fully embedded within the parent object.

```
{
        "@type": "domainName",
        "name": "name.example",
        "nameservers": [
            {
                "@type": "host",
                "hostName": "ns1.name.example",
                "provMetadata": {
                    "@type": "provMetadata",
                    "repositoryId": "NS1-REP",
                    "spClientId": "ClientX"
                },
                "status": [ { "@type": "hostStatus", "label": "ok" } ],
                "dns": {
                    "@type": "dnsData",
                    "records": [
                        {
                            "@type": "dnsRecord",
                            "name": "@",
                            "type": "ns",
                            "rdata": { "nsdname": "ns1.name.example." }
                        },
                        {
                            "@type": "dnsRecord",
                            "name": "ns1.name.example.",
                            "type": "a",
                            "rdata": { "address": "192.0.2.1" }
                        }
                    ]
                }
            }
        ]
}
```

4.5.4. Labelled Aggregation

A `LabelledAggregation[Type]` is a relationship between two independent objects where each association carries a string label. Multiple associations with the same label are allowed.

`LabelledAggregation[Type]` with cardinality `0+` MUST be represented as a JSON array of objects. Each object in the array MUST contain a `label` property (string) alongside the identifier of the referenced object. The object MUST include at least the Unique Identifier of the referenced object type. Other data elements of the referenced object type MAY be included as needed to provide additional context for the client, but are not required. The JSON Schema MUST allow for the presence of these additional fields.

Example: domain contacts (LabelledAggregation[Contact Object]):

```
"contacts": [
    {
        "label": "admin",
        "object": {
            "@type": "contact",
            "id": "ABC-8013"
        }
    },
    {
        "label": "tech",
        "object": {
            "@type": "contact",
            "id": "ABC-8014"
        }
     }
]
```

4.5.5. Dictionary Aggregation

A `DictionaryAggregation[Type]` is a relationship between two independent objects where each association carries a unique string label that serves as a dictionary key.

`DictionaryAggregation[Type]` MUST be represented as a JSON object where each key is the unique label and the corresponding value is the referenced object, the object MUST include at least the Unique Identifier of the referenced object type. Other data elements of the referenced object type MAY be included as needed to provide additional context for the client, but are not required. The JSON Schema MUST allow for the presence of these additional fields.

Example: domain contacts keyed by unique role (DictionaryAggregation[Contact Object]):

```
"contacts": {
    "admin": {
        "@type": "contact",
        "id": "ABC-8013"
    },
    "tech": {
        "@type": "contact",
        "id": "ABC-8014"
    }
}
```

4.5.6. Labelled Composition

A `LabelledComposition[Type]` is a parent-child relationship where each embedded child carries a string label. Multiple instances with the same label are allowed.

`LabelledComposition[Type]` with cardinality `0+` MUST be represented as a JSON array of embedded objects. Each object in the array MUST contain a `label` property alongside the data elements of the composed type.

Example: remarks (LabelledComposition[Remark Object]):

```
"remarks": [
    {
        "label": "public",
        "object": {
            "@type": "remark",
            "description": "This domain is used for test purposes."
        }
    },
    {
        "label": "private",
        "object": {
            "@type": "remark",
            "description": "Internal note for the sponsoring client."
        }
    }
]
```

4.5.7. Dictionary Composition

A `DictionaryComposition[Type]` is a parent-child relationship where each embedded child carries a unique string label used as a dictionary key.

`DictionaryComposition[Type]` MUST be represented as a JSON object where each key is the unique label and the corresponding value is the fully embedded child object.

Example: remarks keyed by scope (DictionaryComposition[Remark Object]):

```
"remarks": {
    "public": {
        "@type": "remark",
        "description": "This domain is used for test purposes."
    },
    "private": {
        "@type": "remark",
        "description": "Internal note for the sponsoring client."
    }
}
```

4.6. Object Identifier Rules

When a resource or component object is referenced by identifier (for example in an aggregation), the identifier MUST be represented as a JSON string using the value of the object's primary identifier data element.

When a resource or component object is embedded (as in a composition), all data elements of the object MUST be represented as properties of a JSON object according to the rules of this section.

4.7. External Type Embedding Rules

RPP data objects may include data elements whose types are defined by external specifications (e.g., JSContact in [RFC9553]). When embedding a value of an externally defined type into a JSON representation, the following rules apply, in order of preference:

If the external type has a native JSON representation (i.e., the external specification defines how the type is represented using the JSON format), the value MUST be embedded as a valid JSON element using the native JSON representation defined by that specification.

If the external type does not have a native JSON representation but has a native UTF-8 encoded text representation, the value MUST be embedded as a JSON `string` using the UTF-8 encoding.

If the external type has neither a native JSON nor a native UTF-8 text representation, the binary representation of the value MUST be Base64-encoded ([RFC4648], Section 4) and embedded as a JSON `string`.

4.8. Process Data Embedding Rules

As defined in [I-D.ietf-rpp-core], a uniform interface operation MAY require process data in addition to the object representation data. This section defines how such process data MUST be represented in JSON when transmitted together with the object representation in a single request body.

Process data accompanying a uniform interface operation MUST be represented using the `processes` data element of the owning resource object (see Processes Object), following the Aggregation rules for the corresponding process type (for example `createProcess` for creation-specific inputs). Each embedded process object MUST be a valid JSON representation of the process object. This applies uniformly regardless of whether the resource object already exists.

Example: a domain create request body carrying a two-year registration period as process data embedded in a `createProcess` process object under `processes`:

```
{
    "@type": "domainName",
    "name": "example.example",
    "nameservers": [
        { "@type": "host", "hostName": "ns1.example.example" },
        { "@type": "host", "hostName": "ns2.example.example" }
    ],
    "processes": {
        "@type": "processes",
        "createProcess": [
            {
                "@type": "createProcess",
                "period": {
                    "@type": "period",
                    "value": 2,
                    "unit": "y"
                }
            }
        ]
    }
}
```

4.9. JSON Schema Definition Rules

Each RPP component object and resource object MUST have a corresponding JSON Schema definition. Object definitions MUST be placed in the `$defs` keyword of the JSON Schema document.

Identifier fields MUST use `"type": "string"` in JSON Schema.

Enumeration constraints on string fields MUST be expressed using the `"enum"` keyword in JSON Schema.

Example (Transfer Status enum):

```
"trStatus": {
    "type": "string",
    "enum": ["pending", "clientApproved", "clientCancelled",
             "clientRejected", "serverApproved", "serverCancelled"]
}
```

Each JSON Schema definition for an RPP object MUST include a `"required"` array listing all data elements with cardinality `1` or `1+`.

4.9.1. Undeclared Properties

JSON Schema definitions for extendible RPP objects MUST NOT use `"additionalProperties": false` or `"unevaluatedProperties": false`. However, before validation, schemas on every property level MUST be enriched with `"unevaluatedProperties": false` property to prevent the presence of undeclared properties in JSON instances. JSON Schemas for Object type MAY use `"additionalProperties": true` to allow for free key definition.

4.9.2. Type Property

Every RPP object representation MUST include a `"@type"` property whose value is the object's identifier as registered in the IANA RPP Data Object Registry. This property enables identification and allows clients and servers to unambiguously determine the type of an object. The `"@type"` property MUST be included in the JSON Schema `"properties"` object for each RPP object definition with a `"const"` constraint fixing the value to the object's registered identifier. The `"@type"` property MUST be listed in the `"required"` array of the corresponding JSON Schema definition.

Example (Domain Name Data Object):

```
{
    "@type": "domainName",
    "name": "example.example"
}
```

4.9.3. Authorisation Information in Requests

When a transfer request or other operation requires authorisation information (e.g., EPP-style authinfo), the client MUST NOT include the `authInfo` object in the JSON request body. Instead, the client MUST convey the authorisation information using the `RPP-Authorization` HTTP request header as defined in [I-D.ietf-rpp-core]. Servers MUST reject any request that includes an `authInfo` object in the JSON body with an appropriate error response.

4.10. JSON Schema Composition Rules

The JSON Schema definitions of all RPP objects MUST be located in a single shared `$defs` object at the top level of the RPP schema document. A schema that refers to another definition MUST use the `$ref` keyword with a JSON Pointer fragment to the definition in the shared `$defs` object.

The key of a definition in the shared `$defs` object MUST be formed as follows, using the Identifiers of the data model unchanged:

- Data Object, Process Object, or Component Object: its Identifier, followed by a `.` and the name of the view. An object that extends another object uses its own Identifier, not the Identifier of the object it extends.

The view names are:

- `create`: the request content of the Create operation.
- `read`: the object representation returned in responses.
- `update`: the request content of the Update operation.
- `reference`: the limited representation of an aggregated object (see Aggregation), containing `@type` and the object's Unique Identifier.
- `query`: the response content of the Query operation, typically an array of `read` views.
- The Identifier of the operation: the request content of any other operation.

- The Identifier of the operation followed by `.output`: the response content of an operation, other than Read and Query, whose output is not an existing view of the object.

A Component Object has a `read` view. A Component Object that has at least one data element with mutability `create-only` or `read-write` also has a `create` view. A Component Object has no other views.

A view that holds request content MUST reference the `create` view of an embedded Component Object. A view that holds response content MUST reference the `read` view of an embedded Component Object.

Each key MUST be unique within the shared `$defs` object. Common Data Types and types defined by an external specification are not subject to this rule. They MAY use a key that is not derived from an RPP Identifier.

A schema that refers to another component object or data object MUST use the `$ref` keyword with a JSON Pointer fragment to the definition in the shared `$defs` object, for example `{ "$ref": "#/$defs/period.create" }`.

The final RPP schema document, containing the shared `$defs` object with all sub-schemas, MUST have an `$id` keyword whose value is an absolute URI without a fragment, serving as the stable identifier of the complete RPP schema document.

Example (structure of the final RPP schema document):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "urn:ietf:params:rpp:schema:json-1.0",
  "$defs": {
    "period.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "period" },
        "value": { "type": "integer" },
        "unit":  { "type": "string", "enum": ["y", "m"] }
      },
      "required": ["@type", "value", "unit"]
    },
    "domainCreateProcess.create": {
      "type": "object",
      "properties": {
        "@type":     { "type": "string", "const": "domainCreateProcess" },
        "period":    { "$ref": "#/$defs/period.create" }
      },
      "required": ["@type"]
    }
  }
}
```

4.10.1. Status Objects

Each status object has a status values table whose "Set By" column identifies the entity that adds and removes each status value: `client`, `server`, or `client, server`. In this section, a client-settable status value is a value whose "Set By" column contains `client`.

The `read` view of a status object lists all of its status values in the `"enum"` of `"label"`. Its `create` view lists only the client-settable status values in the `"enum"` of `"label"`.

When a status object has no client-settable status value, the `create` and `update` views of the Data Object MUST NOT contain the status data element.

When the status values of a status object state that a client is not required to provide a status value when creating or updating the object, the status data element MUST NOT be listed under `"required"` and MUST NOT carry `"minItems"` in the `create` and `update` views, notwithstanding Cardinality Rules and JSON Schema Definition Rules.

When the status values of a status object state that a server MUST assign at least one status value to the object, the status data element MUST be listed under `"required"` in the `read` view, with `"minItems": 1` when it is an array.

4.10.2. Inheritance

RPP Data Objects and Component Objects MAY inherit the data elements and operations of another object. The JSON Schema for an object must include all the inherited data elements and constraints from its base object, using a flat structure where all properties are defined at the top level of the schema, rather than nested under an `"allOf"` or `"anyOf"` construct.

Example: a Shape Object with the data elements `name` (cardinality `1`) and `colour` (cardinality `0-1`), and a Circle Object that extends it with the additional data element `radius` (cardinality `1`).

JSON Schema of the base Shape Object:

```
{
  "$defs": {
    "shape": {
      "type": "object",
      "properties": {
        "@type":  { "type": "string", "const": "shape" },
        "name":   { "type": "string" },
        "colour": { "type": "string" }
      },
      "required": ["@type", "name"]
    }
  }
}
```

JSON Schema of the Circle Object, which defines the inherited `name` and `colour` properties at the top level alongside its own `radius` property, without referencing the Shape Object schema using `"allOf"`:

```
{
  "$defs": {
    "circle": {
      "type": "object",
      "properties": {
        "@type":  { "type": "string", "const": "circle" },
        "name":   { "type": "string" },
        "colour": { "type": "string" },
        "radius": { "type": "integer" }
      },
      "required": ["@type", "name", "radius"]
    }
  }
}
```

Example Circle Object instance:

```
{
    "@type": "circle",
    "name": "my-circle",
    "colour": "red",
    "radius": 10
}
```

4.10.3. RPP Profiles and Validation

RPP profiles, such as the EPP Compatibility Profile, may impose additional constraints on top of the base RPP data model. These additional constraints MUST be enforced by implementations through validation rules that go beyond what can be expressed in JSON Schema. Such validation rules MUST be clearly documented in the profile specification and implemented by both clients and servers when operating under that profile. For example, the EPP Compatibility Profile requires that certain fields be present in specific object types, and that certain identifier fields conform to EPP syntax rules. These constraints cannot be fully captured in JSON Schema and therefore require additional validation logic in implementations.

5. Update Rules

JSON representations for update operations follow specific rules to ensure consistency and correctness. There are two types of update operations: full updates and partial updates.

5.1. Full

A full update operation replaces the entire state of a resource object with the new representation provided in the request, all required properties MUST be included in the new representation.

5.2. Partial

A partial update operation allows clients to update specific properties of a resource object, while leaving other properties unchanged. A partial update MUST use JSON Patch [RFC6902], extended with a `match` property for matching array elements. The `match` property is not part of [RFC6902]; it is introduced and defined in this document (see Section 5.2.1).

The request body of a partial update is a JSON array of patch operations, see Section 5.3. Each patch operation consists of an `op` (the operation), a `path` (a JSON Pointer to the property the operation targets), a `value` (the new value, for `add` and `replace`), and optionally a `match` (to select array elements). When the `path` points to an existing primitive type property, the `add` operation has the same semantics as `replace` and will overwrite the existing value.

5.2.1. Matching

The `match` property is used to identify elements in an array of objects. It is a JSON object that contains key-value pairs corresponding to the properties of the array elements. The server uses the `match` property to locate the elements in the array that should be removed or replaced. The `match` property MUST NOT be used when the `path` does not point to an array, and MUST NOT be used for an array of primitive types. An array of primitive types is changed by replacing the array as a whole, or by adding a value without `match`.

The following rules apply to matching and depend on the type of the property in the `match` object:

- A primitive type property (e.g. string, number or boolean) matches if the target object property has the same type and the same value.
- An array of primitive type property (e.g. array of string, number or boolean) matches if the target array contains elements of the same type and both sets of values are identical. The ordering of elements does not affect matching.
- An object type is considered a match when the value of every property in the `match` object matches the value of the property with the same name in the target object. Properties of the target object that are not named in the `match` object are not compared.
- The server MUST NOT match nested data structures. The `match` property is only used for matching elements in the immediate array specified by the `path` property of the JSON Patch operation.
- The server MUST reject a `match` that is not a JSON object, or that contains a value that cannot be compared using the rules above.

Example domain object for matching an array element with a specific key-value pair:

```
{
    "@type": "domainName",
    "name": "example.example",
    "contacts": [
        { "label": "admin", "object": { "@type": "contact", "id":
"REG-100" } },
        { "label": "tech", "object": { "@type": "contact", "id":
"REG-200" } },
        { "label": "tech", "object": { "@type": "contact", "id":
"REG-300" } }
    ]
}
```

Example `match` object for matching the array element with the "tech" label and the contact ID "REG-200" in the `contacts` array of the domain object above:

```
{
"match": {
      "label": "tech",
      "object": { "@type": "contact", "id": "REG-200" }
    }
}
```

Example object for matching an array element that has a property containing an array of primitive types, here the `tld` property of the elements of the `idnLgr` array:

```
{
    "@type": "discovery",
    "idnLgr": [
        {
            "@type": "discoveryIdnLgr",
            "tld": ["example", "org"],
            "name": "example-latn-1.0"
        },
        {
            "@type": "discoveryIdnLgr",
            "tld": ["example"],
            "name": "example-cyrl-1.0"
        }
    ]
}
```

Example `match` object for matching the elements of the `idnLgr` array above for which the `tld` array contains the value "org" and "example". The order of the values in `tld` is ignored and only the first element matches, because the second element does not contain "org":

```
{
"match": {
      "tld": ["org", "example"]
    }
}
```

5.2.2. Processing Steps

The server MUST process the patch operations in the order in which they appear in the request body. A partial update is atomic, meaning that if any of the operations fails or is rejected, the server MUST reject the complete request with an appropriate error response that identifies the patch operation that failed.

For each patch operation the server MUST perform the following steps. Figure 1 shows the steps for a single patch operation, Figure 2 shows the handling of a primitive type value or an object, and Figure 3 shows the handling of an array.

1. Check the operation: the `op` must be one of "add", "remove" or "replace". JSON Patch operations other than these are not allowed.
2. Resolve the path: the `path` must point to an existing property in the JSON object or a non-existing but allowed property defined in the JSON Schema of the object.
3. Determine the type of the target that the `path` points to:
   - A primitive type value (for example a string, number or boolean), or an object: continue with step 4.
   - An array: continue with step 5.
4. Apply the operation to a primitive type value or an object:
   - For "remove", the field is removed.
   - For "add" and "replace", the `value` property must be present.
     - If the target is a primitive type value, the type of `value` must match the type of the target field.
     - If the target is an object, `value` must be a valid object, the `@type` property of `value` must match the `@type` property of the target object or be a supertype of the target object's `@type`.
     - The field is then set to `value`.
5. Apply the operation to an array:
   - If `match` is present, the operation targets the elements identified by `match`:
     - The "add" operation is not allowed to be used together with `match`.
     - If no element matches, the operation fails.
     - For "remove", all matching elements are removed from the array.
     - For "replace", if more than one element matches, the operation is ambiguous and fails. If exactly one element matches, it is replaced by `value`.
   - If `match` is absent, the operation targets the array property as a whole:
     - For "add", `value` is appended to the array as a new element.
     - For "replace", the entire array is replaced with the array provided in `value`.
     - For "remove", the array property is removed entirely.

6. After the last operation has been applied, the resulting resource object must satisfy the JSON Schema and the constraints of the resource object that cannot be enforced by the JSON Schema.

If a step fails because no element matches, the server MUST return an error response indicating that the specified element could not be found.



Figure 1: Partial update, steps for each patch operation

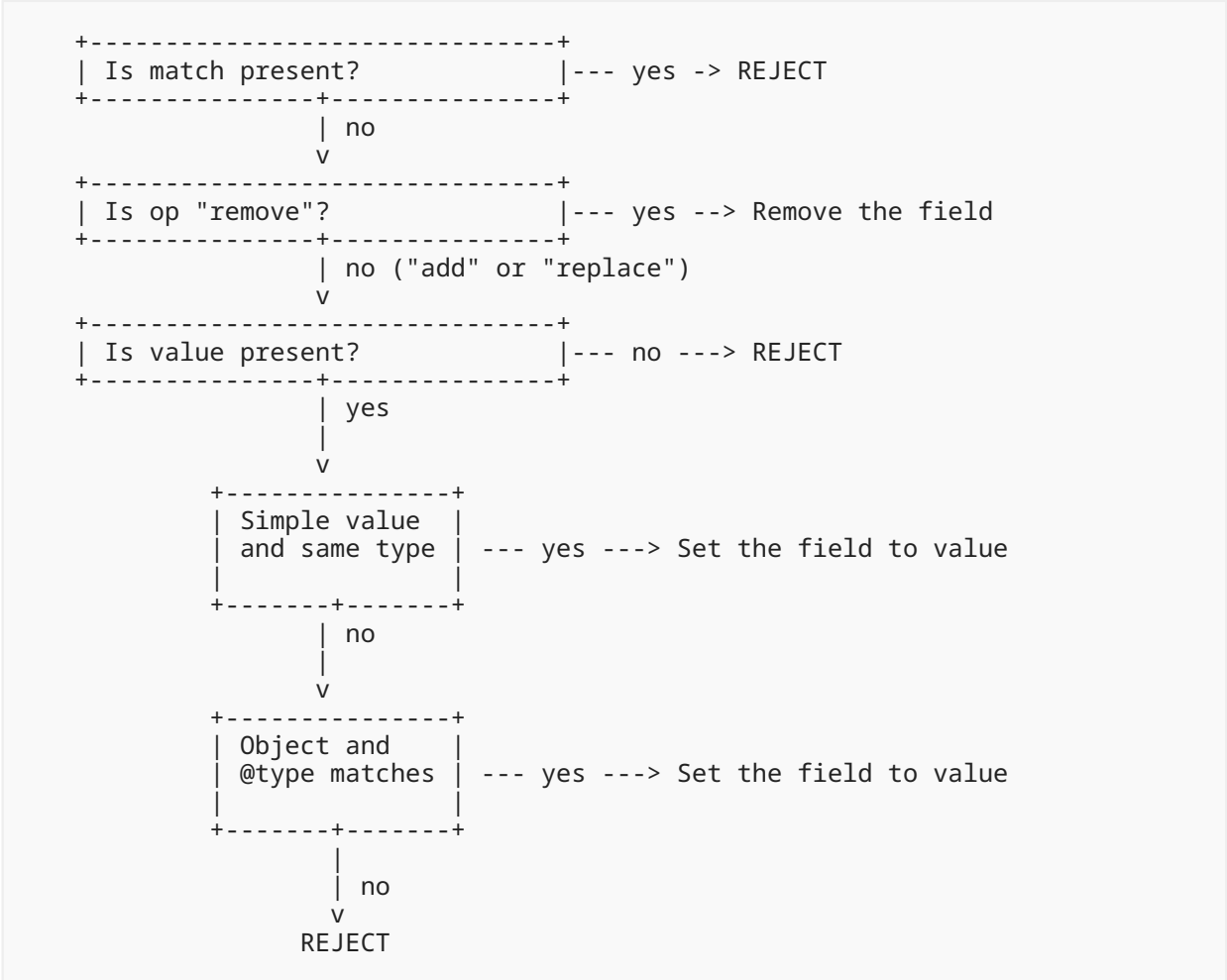
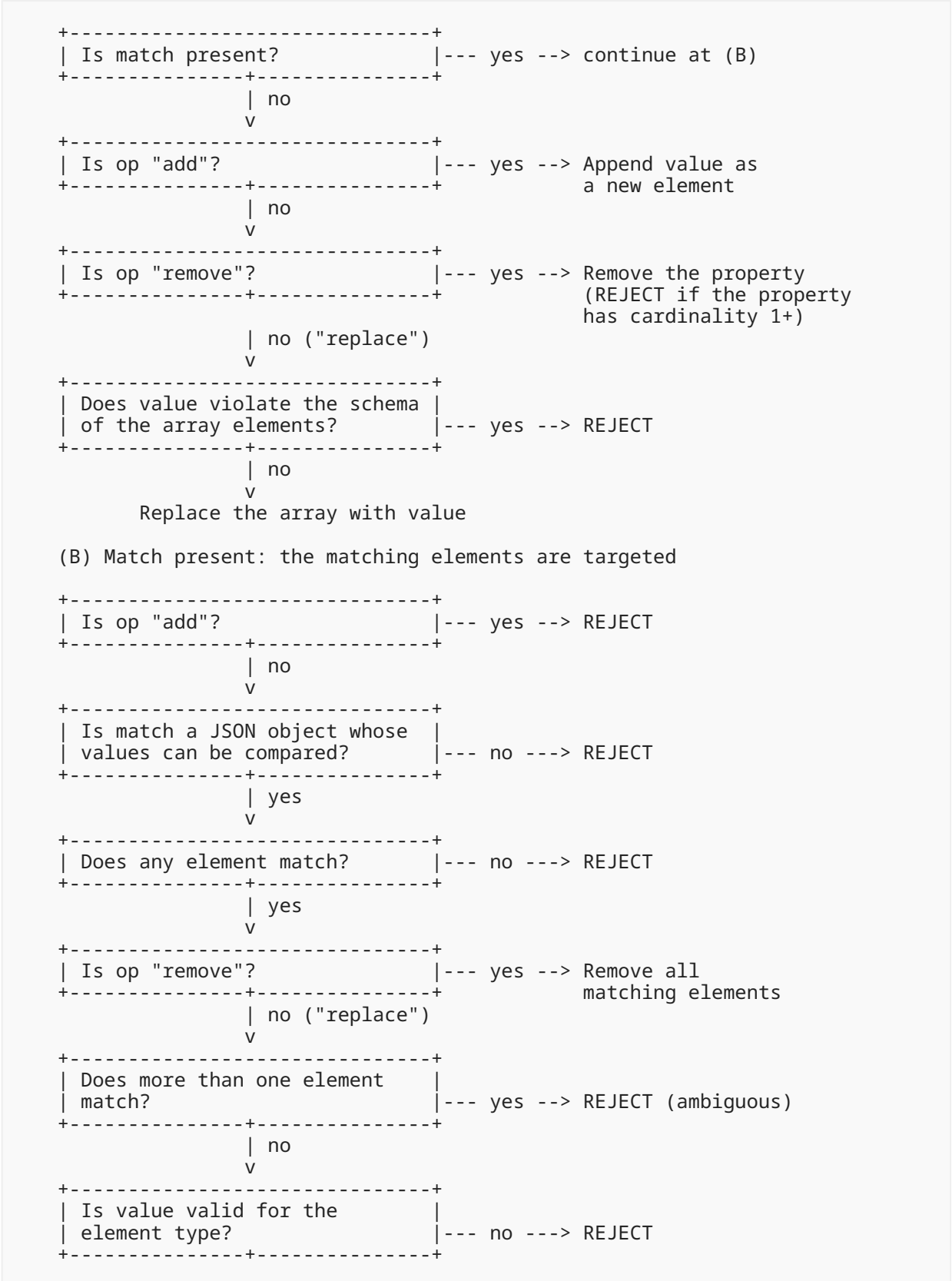


Figure 2: Partial update, primitive type value or object


Is match present?
yes --> continue at (B)
no
Is op "add"?
yes --> Append value as a new element
no
Is op "remove"?
yes --> Remove the property (REJECT if the property has cardinality 1+)
no ("replace")
Does value violate the schema of the array elements?
yes --> REJECT
no
Replace the array with value
(B) Match present: the matching elements are targeted
Is op "add"?
yes --> REJECT
no
Is match a JSON object whose values can be compared?
no ---> REJECT
yes
Does any element match?
no ---> REJECT
yes
Is op "remove"?
yes --> Remove all matching elements
no ("replace")
Does more than one element match?
yes --> REJECT (ambiguous)
no
Is value valid for the element type?
no ---> REJECT

```
                    | yes
                    v
          Replace the element with value
```

Figure 3: Partial update, array

5.3. Schema

A partial update request body MUST be a JSON array of patch operation objects. Each operation MUST include an `op` and a `path` property. The `value` property MUST be present for `add` and `replace` operations. The `match` property MAY be present when the `path` points to an array property, to identify the array elements to operate on. When `match` is absent, the operation targets the array property as a whole (see step 5 in Section 5.2.2).

The `value` property is intentionally left unconstrained (`{}`) in the JSON Schema below, since the JSON type it MUST hold depends on the target `path`: it can be a JSON object (e.g. when replacing a single object property or a single matched array element), a JSON array (e.g. when replacing an array property as a whole), or a primitive type such as a string, number, or boolean.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/partialUpdate",
  "$defs": {
    "partialUpdate": {
      "type": "array",
      "items": { "$ref": "#/$defs/patchOperation" },
      "minItems": 1
    },
    "patchOperation": {
      "oneOf": [
        { "$ref": "#/$defs/patchAdd" },
        { "$ref": "#/$defs/patchRemove" },
        { "$ref": "#/$defs/patchReplace" }
      ]
    },
    "patchAdd": {
      "type": "object",
      "properties": {
        "op":    { "type": "string", "const": "add" },
        "path":  { "type": "string" },
        "value": {}
      },
      "required": ["op", "path", "value"],
      "additionalProperties": false
    },
    "patchRemove": {
      "type": "object",
      "properties": {
        "op":    { "type": "string", "const": "remove" },
        "path":  { "type": "string" },
        "match": { "type": "object" }
      },
      "required": ["op", "path"],
      "additionalProperties": false
    },
    "patchReplace": {
      "type": "object",
      "properties": {
        "op":    { "type": "string", "const": "replace" },
        "path":  { "type": "string" },
        "match": { "type": "object" },
        "value": {}
      },
      "required": ["op", "path", "value"],
      "additionalProperties": false
    }
  }
}
```

5.3.1. Examples

Example JSON document used by partial update examples below:

```
{
    "@type": "domainName",
    "name": "example.example",
    "provMetadata": {
        "@type": "provMetadata",
        "repositoryId": "EXAMPLE1-REP",
        "spClientId": "ClientX",
        "crClientId": "ClientX",
        "crDate": "1999-04-03T22:00:00.0Z"
    },
    "nameservers": [
        { "@type": "host", "hostName": "ns1.example.example" },
        { "@type": "host", "hostName": "ns2.example.example" }
    ],
    "registrant": { "@type": "contact", "id": "JD-1234" },
    "contacts": [
        { "label": "admin", "object": { "@type": "contact", "id":
"SH-8013" } },
        { "label": "tech", "object": { "@type": "contact", "id":
"SH-8013" } }
    ],
    "status": [
        { "@type": "domainStatus", "label": "ok" }
    ],
    "expiryDate": "2027-01-01T00:00:00.0Z"
}
```

Example of a partial update request to change the registrant of a domain name from "JD-1234" to "JD-1235":

```
[
  {
    "op": "replace",
    "path": "/registrant",
    "value": {
      "@type": "contact",
      "id": "JD-1235"
    }
  }
]
```

Example of a partial update request to change the status of a domain name from "ok" to "clientUpdateProhibited":

```
[
  {
    "op": "replace",
    "path": "/status",
    "match": {
      "label": "ok"
    },
    "value": {
      "@type": "domainStatus",
      "label": "clientUpdateProhibited"
    }
  }
]
```

Example of a partial update request for adding an additional tech contact to a domain name:

```
[
  {
    "op": "add",
    "path": "/contacts",
    "value": {
      "label": "tech",
      "object": {
        "@type": "contact",
        "id": "SH-8014"
      }
    }
  }
]
```

Example of a partial update request replacing the entire `nameservers` array of a domain name in a single operation, with no "match" property present:

```
[
  {
    "op": "replace",
    "path": "/nameservers",
    "value": [
      { "@type": "host", "hostName": "ns1.example.example" },
      { "@type": "host", "hostName": "ns3.example.example" }
    ]
  }
]
```

6. JSON Schema Definitions

This section provides normative JSON Schema definitions for RPP component objects and resource objects. All schemas use JSON Schema draft 2020-12 [I-D.ietf-jsonschema-json-schema].

6.1. Common Data Types Schemas

This section defines shared data types that are based on the primitive data types above and are reused across multiple data object definitions.

6.1.1. Identifier

Identifiers are character strings with a specified minimum length, a specified maximum length, and a specified format outlined in Section 2.8 of [RFC5730]. Identifiers for certain object types MAY have additional constraints imposed either by server policy, object-specific specifications, or both.

In JSON, an Identifier MUST be represented as a `string` value conforming to the `roidType` syntax defined in Section 2.8 of [RFC5730]: one to eight word characters optionally preceded by an underscore, followed by a hyphen and one or more word characters, with a maximum total length of 89 characters.

```
{
  "$defs": {
    "identifier": {
      "type": "string",
      "pattern": "^(\\w|_){1,8}-\\w{1,}$",
      "maxLength": 89
    }
  }
}
```

6.1.2. Client Identifier

Client identifiers are character strings with a specified minimum length, a specified maximum length, and a specified format. Client identifiers use the `clIDType` syntax described in [RFC5730].

In JSON, a Client Identifier MUST be represented as a `string` value.

```
{
  "$defs": {
    "clientIdentifier": {
      "type": "string",
      "minLength": 3,
      "maxLength": 16,
      "pattern": "^\\S+( \\S+)*$"
    }
  }
}
```

6.1.3. Phone Number

Telephone number syntax is derived from structures defined in [ITU.E164.2005]. Telephone numbers described in this specification are character strings that MUST begin with a plus sign ("+", ASCII value 0x002B), followed by a country code defined in [ITU.E164.2005],

followed by a dot (".", ASCII value 0x002E), followed by a sequence of digits representing the telephone number. An optional "x" (ASCII value 0x0078) separator with additional digits representing extension information can be appended to the end of the value.

In JSON, a Phone Number MUST be represented as a `string` value conforming to the pattern described above.

```
{
  "$defs": {
    "phoneNumber": { "type": "string", "pattern": "^\\+[0-9]{1,3}\\.[0-9]
{1,14}(x[0-9]+)?$" }
  }
}
```

6.2. Component Objects Schemas

6.2.1. Period Object

Read schema:

```
{
  "$defs": {
    "period.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "period", "readOnly": true },
        "value": { "type": "integer", "minimum": 1, "maximum": 99 },
        "unit": { "type": "string", "enum": ["y", "m"] }
      },
      "required": ["@type", "value", "unit"]
    }
  }
}
```

Create schema:

```
{
  "$defs": {
    "period.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "period" },
        "value": { "type": "integer", "minimum": 1, "maximum": 99 },
        "unit": { "type": "string", "enum": ["y", "m"] }
      },
      "required": ["@type", "value", "unit"]
    }
  }
}
```

6.2.2. Provisioning Metadata Object

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `upClientId` and `upDate` MUST NOT be present if the object has never been modified.
- `trDate` MUST NOT be present if the object has never been transferred.
- In the EPP Compatibility Profile, `repositoryId` MUST be provided.

Read schema:

```
{
  "$defs": {
    "provMetadata.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "provMetadata", "readOnly":
true },
        "repositoryId": { "$ref": "#/$defs/identifier", "readOnly": true },
        "spClientId": { "$ref": "#/$defs/clientIdentifier", "readOnly":
true },
        "crClientId": { "$ref": "#/$defs/clientIdentifier", "readOnly":
true },
        "crDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "upClientId": { "$ref": "#/$defs/clientIdentifier", "readOnly":
true },
        "upDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "trDate": { "type": "string", "format": "date-time", "readOnly":
true }
      },
      "required": ["@type", "spClientId"]
    }
  }
}
```

6.2.3. Status Objects

Each RPP Data Object type that carries a status uses a type-specific status object. The `create` view of a status object holds only the status values that a client can set (see Section 4.10.1).

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- A server MUST keep the status values it has set when it processes a create or update request; their absence from the `"status"` of the request does not remove them. A server MAY alter or override client-settable status values, subject to local server policy.
- A server MUST reject a request whose `"status"` array contains a combination of label values prohibited by the status exclusion rules for that object type.
- A `"status"` array MUST NOT contain the same `"label"` value more than once.

Extensions MAY define additional `"label"` values for a status object; such values are added to the `"enum"` of the corresponding status object in the effective schema, as described in Section 7.

6.2.3.1. Domain Name Status Object

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `"due"`: Servers MAY restrict the ability of clients to set or update this value.
- Label values with the prefix `"rgp"` MUST only appear when the server supports the Redemption Grace Period (RGP) for the domain name.
- The labels `"rgpRedemptionPeriod"`, `"rgpPendingRestore"`, and `"rgpPendingDelete"` MUST only appear together with the `"pendingDelete"` label.

Read schema:

```
{
  "$defs": {
    "domainStatus.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "domainStatus", "readOnly":
true },
        "label": {
          "type": "string",
          "enum": [
            "clientDeleteProhibited",
            "serverDeleteProhibited",
            "clientHold",
            "serverHold",
            "clientRenewProhibited",
            "serverRenewProhibited",
            "clientTransferProhibited",
            "serverTransferProhibited",
            "clientUpdateProhibited",
            "serverUpdateProhibited",
            "inactive",
            "ok",
            "pendingCreate",
            "pendingDelete",
            "pendingRenew",
            "pendingTransfer",
            "pendingUpdate",
            "rgpAddPeriod",
            "rgpAutoRenewPeriod",
            "rgpRenewPeriod",
            "rgpTransferPeriod",
            "rgpRedemptionPeriod",
            "rgpPendingRestore",
            "rgpPendingDelete"
          ],
          "readOnly": true
        },
        "reason": { "type": "string", "readOnly": true },
        "due": { "type": "string", "format": "date-time" }
      },
      "required": ["@type", "label"]
    }
  }
}
```

Create schema (status values that a client can set):

```
{
  "$defs": {
    "domainStatus.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "domainStatus" },
        "label": {
          "type": "string",
          "enum": [
            "clientDeleteProhibited",
            "clientHold",
            "clientRenewProhibited",
            "clientTransferProhibited",
            "clientUpdateProhibited"
          ],
          "writeOnly": true
        },
        "reason": { "type": "string", "writeOnly": true },
        "due": { "type": "string", "format": "date-time" }
      },
      "required": ["@type", "label"]
    }
  }
}
```

6.2.3.2. Host Status Object

Read schema:

```
{
  "$defs": {
    "hostStatus.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "hostStatus", "readOnly":
true },
        "label": {
          "type": "string",
          "enum": [
            "clientDeleteProhibited",
            "serverDeleteProhibited",
            "clientUpdateProhibited",
            "serverUpdateProhibited",
            "linked",
            "ok",
            "pendingCreate",
            "pendingDelete",
            "pendingTransfer",
            "pendingUpdate"
          ],
          "readOnly": true
        },
        "reason": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "label"]
    }
  }
}
```

Create schema (status values that a client can set):

```
{
  "$defs": {
    "hostStatus.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "hostStatus" },
        "label": {
          "type": "string",
          "enum": ["clientDeleteProhibited", "clientUpdateProhibited"],
          "writeOnly": true
        },
        "reason": { "type": "string", "writeOnly": true }
      },
      "required": ["@type", "label"]
    }
  }
}
```

6.2.3.3. Contact Status Object

Read schema:

```
{
  "$defs": {
    "contactStatus.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "contactStatus", "readOnly":
true },
        "label": {
          "type": "string",
          "enum": [
            "clientDeleteProhibited",
            "serverDeleteProhibited",
            "clientTransferProhibited",
            "serverTransferProhibited",
            "clientUpdateProhibited",
            "serverUpdateProhibited",
            "linked",
            "ok",
            "pendingCreate",
            "pendingDelete",
            "pendingTransfer",
            "pendingUpdate"
          ],
          "readOnly": true
        },
        "reason": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "label"]
    }
  }
}
```

Create schema (status values that a client can set):

```
{
  "$defs": {
    "contactStatus.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "contactStatus" },
        "label": {
          "type": "string",
          "enum": ["clientDeleteProhibited", "clientTransferProhibited",
"clientUpdateProhibited"],
          "writeOnly": true
        },
        "reason": { "type": "string", "writeOnly": true }
      },
      "required": ["@type", "label"]
    }
  }
}
```

6.2.3.4. Organisation Role Status Object

Read schema:

```
{
  "$defs": {
    "organisationRoleStatus.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "organisationRoleStatus",
"readOnly": true },
        "label": {
          "type": "string",
          "enum": ["clientLinkProhibited", "serverLinkProhibited", "linked",
"ok"],
          "readOnly": true
        },
        "reason": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "label"]
    }
  }
}
```

Create schema (status values that a client can set):

```
{
  "$defs": {
    "organisationRoleStatus.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "organisationRoleStatus" },
        "label": { "type": "string", "enum": ["clientLinkProhibited"],
"writeOnly": true },
        "reason": { "type": "string", "writeOnly": true }
      },
      "required": ["@type", "label"]
    }
  }
}
```

6.2.3.5. Organisation Status Object

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- The `"hold"` and `"terminated"` labels are server-managed when the organisation has no parent organisation; otherwise they MAY be client-managed, subject to server policy.
- The `"status"` array MUST contain exactly one of the labels `"pendingCreate"`, `"ok"`, `"hold"`, or `"terminated"`.

Read schema:

```
{
  "$defs": {
    "organisationStatus.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "organisationStatus",
"readOnly": true },
        "label": {
          "type": "string",
          "enum": [
            "ok",
            "hold",
            "terminated",
            "linked",
            "clientLinkProhibited",
            "serverLinkProhibited",
            "clientUpdateProhibited",
            "serverUpdateProhibited",
            "clientDeleteProhibited",
            "serverDeleteProhibited",
            "pendingCreate",
            "pendingUpdate",
            "pendingDelete"
          ],
          "readOnly": true
        },
        "reason": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "label"]
    }
  }
}
```

Create schema (status values that a client can set):

```
{
  "$defs": {
    "organisationStatus.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "organisationStatus" },
        "label": {
          "type": "string",
          "enum": [
            "hold",
            "terminated",
            "clientLinkProhibited",
            "clientUpdateProhibited",
            "clientDeleteProhibited"
          ],
          "writeOnly": true
        },
        "reason": { "type": "string", "writeOnly": true }
      },
      "required": ["@type", "label"]
    }
  }
}
```

6.2.3.6. User Status Object

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- The `"pending"` label is server-managed; `"suspended"` and `"deactivated"` are server-managed and MAY additionally be client-managed by a client with permission to manage users.

Read schema:

```
{
  "$defs": {
    "userStatus.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "userStatus", "readOnly":
true },
        "label": {
          "type": "string",
          "enum": ["active", "pending", "suspended", "deactivated"],
          "readOnly": true
        },
        "reason": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "label"]
    }
  }
}
```

Create schema (status values that a client can set):

```
{
  "$defs": {
    "userStatus.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "userStatus" },
        "label": { "type": "string", "enum": ["suspended", "deactivated"],
"writeOnly": true },
        "reason": { "type": "string", "writeOnly": true }
      },
      "required": ["@type", "label"]
    }
  }
}
```

6.2.4. DNS Resource Record Object

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `name` MUST be a syntactically valid DNS host name in zone file string representation. Both absolute FQDNs (with trailing dot) and relative host names are allowed, as well as the `@` symbol representing the domain name itself.
- `type` MUST be a valid string representation of a DNS resource record type as defined in [RFC1035]. Values MUST be converted to lowercase. Allowed values MAY be further constrained by server policy.
- In the EPP Compatibility Profile ([RFC5732]), the following record types MUST be supported: `ns`, `a`, and `aaaa`. With DNSSEC Extension [RFC5910], `ds` and `dnskey` MUST additionally be supported.
- `class`, if present, MUST be chosen from Section 3.2.4 (CLASS values) of [RFC1035]. A client SHOULD omit this element; the server MUST assume `IN` as the default.
- The fields within `rdata` MUST match the expected structure for the given record type (see RDATA structures below).

RDATA structures required in the EPP Compatibility Profile:

- NS records ([RFC1035], Section 3.3.11): `nsdname`
- A records ([RFC1035], Section 3.4.1): `address`
- AAAA records ([RFC3596], Section 2.2): `address`
- DS records ([RFC4034], Section 5, with DNSSEC Extension): `keyTag`, `algorithm`, `digestType`, `digest`
- DNSKEY records ([RFC4034], Section 2, with DNSSEC Extension): `flags`, `protocol`, `algorithm`, `publicKey`

All `rdata` property names MUST be written in camelCase and all values MUST use the string data type.

Read schema:

```
{
  "$defs": {
    "dnsRecord.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "dnsRecord", "readOnly":
true },
        "name": { "type": "string" },
        "class": { "type": "string" },
        "type": { "type": "string" },
        "rdata": { "type": "object", "unevaluatedProperties": true }
      },
      "required": ["@type", "name", "type", "rdata"]
    }
  }
}
```

Create schema:

```
{
  "$defs": {
    "dnsRecord.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "dnsRecord" },
        "name": { "type": "string" },
        "class": { "type": "string" },
        "type": { "type": "string" },
        "rdata": { "type": "object", "unevaluatedProperties": true }
      },
      "required": ["@type", "name", "type", "rdata"]
    }
  }
}
```

6.2.5. DNS Operational Controls Object

The DNS Operational Controls Object contains operational control parameters that a client MAY use to influence server-side DNS behaviour for a set of DNS records. A server MAY ignore these values, e.g. for policy reasons. This structure is aligned with [I-D.simmen-rpp-dns-data].

Read schema:

```
{
  "$defs": {
    "dnsControls.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "dnsControls", "readOnly":
true },
        "ttl": {
          "type": "object",
          "additionalProperties": { "type": "integer", "minimum": 1 },
          "propertyNames": { "pattern": "^[a-z][a-z0-9-]*$" }
        },
        "maxSigLifetime": {
          "type": "object",
          "additionalProperties": { "type": "integer", "minimum": 1 },
          "propertyNames": { "pattern": "^[a-z][a-z0-9-]*$" }
        }
      },
      "required": ["@type"]
    }
  }
}
```

Create schema:

```
{
  "$defs": {
    "dnsControls.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "dnsControls" },
        "ttl": {
          "type": "object",
          "additionalProperties": { "type": "integer", "minimum": 1 },
          "propertyNames": { "pattern": "^[a-z][a-z0-9-]*$" }
        },
        "maxSigLifetime": {
          "type": "object",
          "additionalProperties": { "type": "integer", "minimum": 1 },
          "propertyNames": { "pattern": "^[a-z][a-z0-9-]*$" }
        }
      },
      "required": ["@type"]
    }
  }
}
```

6.2.6. DNS Data Object

The DNS Data Object is a container for DNS resource records and associated operational controls for a provisioned object. This structure groups DNS records together with control parameters that influence server-side DNS behaviour. It is aligned with [I-D.simmen-rpp-dns-data].

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- In the EPP Compatibility Profile with DNSSEC Extension [RFC5910], records of type `ds` and `dnskey` MUST be supported in addition to `ns`, `a`, and `aaaa`. A server MUST support either `ds` or `dnskey` or both. If provided with only `dnskey`, a server MUST calculate the DS record.

Read schema:

```
{
  "$defs": {
    "dnsData.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "dnsData", "readOnly": true },
        "records": {
          "type": "array",
          "items": { "$ref": "#/$defs/dnsRecord.read" }
        },
        "controls": { "$ref": "#/$defs/dnsControls.read" }
      },
      "required": ["@type"]
    }
  }
}
```

Create schema:

```
{
  "$defs": {
    "dnsData.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "dnsData" },
        "records": {
          "type": "array",
          "items": { "$ref": "#/$defs/dnsRecord.create" }
        },
        "controls": { "$ref": "#/$defs/dnsControls.create" }
      },
      "required": ["@type"]
    }
  }
}
```

6.2.7. Authorisation Information Object

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `method` MUST be one of the values registered in the IANA RPP Authorization Method Registry as defined in [I-D.ietf-rpp-core]. In the EPP Compatibility Profile, this value MUST be "authinfo" for standard password-based authorisation.
- The Authorisation Information Object is immutable. When authorisation information changes, a new instance MUST be created rather than modifying the existing one. The value of `authdata` MAY be omitted from read responses, depending on the method and server policy.

Read schema:

```
{
  "$defs": {
    "authInfo.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "authInfo", "readOnly": true },
        "method": { "type": "string", "readOnly": true },
        "authdata": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "method", "authdata"]
    }
  }
}
```

Create schema:

```
{
  "$defs": {
    "authInfo.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "authInfo" },
        "method": { "type": "string", "writeOnly": true },
        "authdata": { "type": "string", "writeOnly": true }
      },
      "required": ["@type", "method", "authdata"]
    }
  }
}
```

6.2.8. Disclose Object

The Disclose Object is used by the `disclose` data element of the Contact Data Object.

> TODO: The Disclose Object is not yet fully defined in the data model. The schema below is a placeholder.

Read schema:

```
{
  "$defs": {
    "disclose.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "disclose", "readOnly": true }
      },
      "required": ["@type"]
    }
  }
}
```

6.2.9. JSContact Card Object

The Contact Data Object uses version 2.0 of JSContact [RFC9982] to represent contact information. The `Contact` object is defined below according to the RPP JSContact Profile [TODO Add Ref].

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `addresses[*].countryCode` MUST be a valid two-character ISO 3166-1 [ISO3166-1] alpha-2 code when present.
- `localizations` MUST be fully expanded; nested PatchObject-style keys (e.g., `"addresses/addr/full"`) are NOT allowed.
- When `phones[*].features` is absent, the number MUST be treated as a voice number.

```
{
  "$defs": {
    "Card": {
      "type": "object",
      "properties": {
        "@type":    { "type": "string", "const": "Card" },
        "version":  { "type": "string", "const": "2.0" },
        "kind":     { "type": "string", "enum": ["individual", "org"] },
        "language": { "type": "string" },
        "name": {
          "type": "object",
          "properties": {
            "full": { "type": "string" }
          }
        },
        "organizations": {
          "type": "object",
          "additionalProperties": {
            "type": "object",
            "properties": {
              "name": { "type": "string" }
            }
          }
        },
        "addresses": {
```

```
          "type": "object",
          "additionalProperties": {
            "type": "object",
            "properties": {
              "full":        { "type": "string" },
              "countryCode": { "type": "string", "pattern": "^[A-Z]{2}$" },
              "components": {
                "type": "array",
                "items": {
                  "type": "object",
                  "properties": {
                    "kind":  { "type": "string", "enum": ["name", "locality",
"region", "postcode"] },
                    "value": { "type": "string" }
                  },
                  "required": ["kind", "value"]
                }
              }
            }
          }
        },
        "phones": {
          "type": "object",
          "additionalProperties": {
            "type": "object",
            "properties": {
              "number":   { "type": "string" },
              "features": {
                "type": "object",
                "additionalProperties": {
                  "type": "boolean"
                }
              }
            },
            "required": ["number"]
          }
        },
        "emails": {
          "type": "object",
          "additionalProperties": {
            "type": "object",
            "properties": {
              "address": { "type": "string", "format": "email" }
            },
            "required": ["address"]
          }
        },
        "links": {
          "type": "object",
          "additionalProperties": {
            "type": "object",
            "properties": {
              "uri":  { "type": "string", "format": "uri" },
              "kind": { "type": "string", "const": "contact" }
            },
            "required": ["uri"]
          }
        },
```

```
        "localizations": { "type": "object" }
      },
      "required": ["@type", "version"]
    }
  }
}
```

6.2.10. Restore Report Object

The Restore Report Object contains the redemption grace period restore report submitted by the sponsoring client as required by the RGP process ([RFC3915]).

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- At least one and at most two `statements` MUST be provided.
- `restoreTime` MAY be omitted when the restore report is submitted inline within the restore request in a single-step process.
- In the EPP Compatibility Profile, `restoreTime` MUST be present as defined in [RFC3915].
- In the EPP Compatibility Profile, exactly two `statements` MUST be present as defined in [RFC3915].

Read schema:

```
{
  "$defs": {
    "restoreReport.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "restoreReport", "readOnly":
true },
        "preData": { "type": "string" },
        "postData": { "type": "string" },
        "deleteTime": { "type": "string", "format": "date-time" },
        "restoreTime": { "type": "string", "format": "date-time" },
        "restoreReason": { "type": "string" },
        "statements": {
          "type": "array",
          "items": { "type": "string" },
          "minItems": 1,
          "maxItems": 2
        },
        "other": { "type": "string" }
      },
      "required": ["@type", "statements"]
    }
  }
}
```

Create schema:

```
{
  "$defs": {
    "restoreReport.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "restoreReport" },
        "preData": { "type": "string" },
        "postData": { "type": "string" },
        "deleteTime": { "type": "string", "format": "date-time" },
        "restoreTime": { "type": "string", "format": "date-time" },
        "restoreReason": { "type": "string" },
        "statements": {
          "type": "array",
          "items": { "type": "string" },
          "minItems": 1,
          "maxItems": 2
        },
        "other": { "type": "string" }
      },
      "required": ["@type", "statements"]
    }
  }
}
```

6.2.11. Organisation Role Object

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `type` MUST only use values registered in the IANA "EPP Organization Role Values" registry ([RFC8543]).

Read schema:

```
{
  "$defs": {
    "organisationRole.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "organisationRole", "readOnly":
true },
        "roleId": { "type": "string" },
        "status": {
          "type": "array",
          "items": { "$ref": "#/$defs/organisationRoleStatus.read" }
        }
      },
      "required": ["@type"]
    }
  }
}
```

Create schema:

```
{
  "$defs": {
    "organisationRole.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "organisationRole" },
        "roleId": { "type": "string" },
        "status": {
          "type": "array",
          "items": { "$ref": "#/$defs/organisationRoleStatus.create" }
        }
      },
      "required": ["@type"]
    }
  }
}
```

6.2.12. Extension Object

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `version` MUST use the Semantic Versioning format.
- `url` MUST be a valid URL and accessible to the client.

Read schema:

```
{
  "$defs": {
    "extension.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "extension", "readOnly":
true },
        "name": { "type": "string", "readOnly": true },
        "id": { "type": "string", "readOnly": true },
        "version": { "type": "string", "readOnly": true },
        "url": { "type": "string", "format": "uri", "readOnly": true }
      },
      "required": ["@type", "name", "id", "version", "url"]
    }
  }
}
```

6.2.13. Discovery

6.2.13.1. Version Object

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `version` MUST use the Semantic Versioning format.

Read schema:

```
{
  "$defs": {
    "discoveryVersion.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "discoveryVersion", "readOnly":
true },
        "version": { "type": "string", "readOnly": true },
        "baseUrl": { "type": "string", "format": "uri", "readOnly": true }
      },
      "required": ["@type", "version", "baseUrl"]
    }
  }
}
```

6.2.13.2. Environment Object

Read schema:

```
{
  "$defs": {
    "discoveryEnvironment.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "discoveryEnvironment",
"readOnly": true },
        "name": { "type": "string", "readOnly": true },
        "status": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "name", "status"]
    }
  }
}
```

6.2.13.3. IDN LGR Object

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `name` MUST be a valid LGR name as listed in the [IDN-Tables] registry.

Read schema:

```
{
  "$defs": {
    "discoveryIdnLgr.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "discoveryIdnLgr", "readOnly":
true },
        "tld": {
          "type": "array",
          "items": { "type": "string" },
          "minItems": 1,
          "readOnly": true
        },
        "name": { "type": "string", "readOnly": true },
        "url": { "type": "string", "format": "uri", "readOnly": true },
        "default": { "type": "boolean", "readOnly": true }
      },
      "required": ["@type", "tld", "name", "url", "default"]
    }
  }
}
```

6.2.13.4. Notice Object

Read schema:

```
{
  "$defs": {
    "discoveryNotice.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "discoveryNotice", "readOnly":
true },
        "pubTime": { "type": "string", "format": "date-time", "readOnly":
true },
        "text": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "pubTime", "text"]
    }
  }
}
```

6.2.14. Processes Object

The Processes Object is a read-only container grouping the currently active Process Objects on the owning resource object, keyed by process type. Each key holds an array of the corresponding Process Object's read representation, per the Aggregation rules.

Read schema:

```
{
  "$defs": {
    "processes.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "processes", "readOnly":
true },
        "transferProcess": {
          "type": "array",
          "items": {
            "oneOf": [
              { "$ref": "#/$defs/transferProcess.reference" },
              { "$ref": "#/$defs/domainTransferProcess.reference" }
            ]
          },
          "readOnly": true
        },
        "renewProcess": {
          "type": "array",
          "items": { "$ref": "#/$defs/renewProcess.reference" },
          "readOnly": true
        },
        "restoreProcess": {
          "type": "array",
          "items": { "$ref": "#/$defs/restoreProcess.reference" },
          "readOnly": true
        },
        "createProcess": {
          "type": "array",
          "items": {
            "oneOf": [
              { "$ref": "#/$defs/createProcess.reference" },
              { "$ref": "#/$defs/domainCreateProcess.reference" }
            ]
          },
          "readOnly": true
        }
      },
      "required": ["@type"]
    }
  }
}
```

6.2.15. Availability

The Availability Objects are the output of the Availability operation. The Base Availability Object is extended by Data Object specific Availability Objects in order to allow independent extensibility.

6.2.15.1. Base Availability Object

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `reason` MUST only be present when `available` is `false`.

Read schema:

```
{
  "$defs": {
    "baseAvailability.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "baseAvailability", "readOnly":
true },
        "available": { "type": "boolean", "readOnly": true },
        "reason": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "available"]
    }
  }
}
```

6.2.15.2. Domain Name Availability Object

Read schema:

```
{
  "$defs": {
    "domainNameAvailability.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "domainNameAvailability",
"readOnly": true },
        "available": { "type": "boolean", "readOnly": true },
        "reason": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "available"]
    }
  }
}
```

6.2.15.3. Contact Availability Object

Read schema:

```
{
  "$defs": {
    "contactAvailability.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "contactAvailability",
"readOnly": true },
        "available": { "type": "boolean", "readOnly": true },
        "reason": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "available"]
    }
  }
}
```

6.2.15.4. Host Availability Object

Read schema:

```
{
  "$defs": {
    "hostAvailability.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "hostAvailability", "readOnly":
true },
        "available": { "type": "boolean", "readOnly": true },
        "reason": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "available"]
    }
  }
}
```

6.2.16. Messages

All message objects are generated by the server and are read-only from the perspective of the receiving client.

The Base Message Object is extended by all other message objects defined in this section. The JSON Schema of each message object repeats the data elements it inherits from the Base Message Object as top-level properties. The value of the `@type` property identifies the concrete message object and enables a client to determine which data elements to expect.

6.2.16.1. Base Message Object

The Base Message Object contains the data elements common to all message objects.

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `objectType` MUST only be used when `objectId` is present.

- `objectId` MUST be a valid unique identifier for the object type specified in `objectType` or empty when `objectType` is not present or empty.

Read schema:

```
{
  "$defs": {
    "baseMessage.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "baseMessage", "readOnly":
true },
        "objectType": { "type": "string", "readOnly": true },
        "objectId": { "type": "string", "readOnly": true }
      },
      "required": ["@type"]
    }
  }
}
```

6.2.16.2. Review Response Object

Read schema:

```
{
  "$defs": {
    "reviewResponseMessage.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "reviewResponseMessage",
"readOnly": true },
        "objectType": { "type": "string", "readOnly": true },
        "objectId": { "type": "string", "readOnly": true },
        "approved": { "type": "boolean", "readOnly": true },
        "clientTransactionId": { "type": "string", "readOnly": true },
        "serverTransactionId": { "type": "string", "readOnly": true },
        "completionDate": { "type": "string", "format": "date-time",
"readOnly": true },
        "message": { "type": "string", "readOnly": true },
        "process": {
          "oneOf": [
            { "$ref": "#/$defs/transferProcess.read" },
            { "$ref": "#/$defs/domainTransferProcess.read" },
            { "$ref": "#/$defs/restoreProcess.read" },
            { "$ref": "#/$defs/renewProcess.read" },
            { "$ref": "#/$defs/createProcess.read" },
            { "$ref": "#/$defs/domainCreateProcess.read" }
          ],
          "readOnly": true
        }
      },
      "required": ["@type", "approved", "serverTransactionId",
"completionDate", "message"]
    }
  }
}
```

6.2.16.3. Transfer Request Message Object

Read schema:

```
{
  "$defs": {
    "transferRequestMessage.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "transferRequestMessage",
"readOnly": true },
        "objectType": { "type": "string", "readOnly": true },
        "objectId": { "type": "string", "readOnly": true },
        "status": { "type": "string", "enum": ["pending"], "readOnly":
true },
        "requestDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "actionDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "process": {
          "oneOf": [
            { "$ref": "#/$defs/transferProcess.read" },
            { "$ref": "#/$defs/domainTransferProcess.read" }
          ],
          "readOnly": true
        }
      },
      "required": ["@type", "status", "requestDate"]
    }
  }
}
```

6.2.16.4. Transfer Outcome Message Object

Read schema:

```
{
  "$defs": {
    "transferOutcomeMessage.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "transferOutcomeMessage",
"readOnly": true },
        "objectType": { "type": "string", "readOnly": true },
        "objectId": { "type": "string", "readOnly": true },
        "trStatus": {
          "type": "string",
          "enum": [
            "clientApproved",
            "clientCancelled",
            "clientRejected",
            "serverApproved",
            "serverCancelled"
          ],
          "readOnly": true
        },
        "requestDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "actionDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "expiryDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "process": {
          "oneOf": [
            { "$ref": "#/$defs/transferProcess.read" },
            { "$ref": "#/$defs/domainTransferProcess.read" }
          ],
          "readOnly": true
        }
      },
      "required": ["@type", "trStatus", "requestDate", "actionDate"]
    }
  }
}
```

6.2.16.5. Expiration Deletion Message Object

Read schema:

```
{
  "$defs": {
    "expirationDeletionMessage.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "expirationDeletionMessage",
"readOnly": true },
        "objectType": { "type": "string", "readOnly": true },
        "objectId": { "type": "string", "readOnly": true },
        "deletionDate": { "type": "string", "format": "date-time",
"readOnly": true }
      },
      "required": ["@type", "deletionDate"]
    }
  }
}
```

6.2.16.6. Auto-Renewal Message Object

Read schema:

```
{
  "$defs": {
    "autoRenewalMessage.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "autoRenewalMessage",
"readOnly": true },
        "objectType": { "type": "string", "readOnly": true },
        "objectId": { "type": "string", "readOnly": true },
        "renewalDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "expiryDate": { "type": "string", "format": "date-time", "readOnly":
true }
      },
      "required": ["@type", "renewalDate", "expiryDate"]
    }
  }
}
```

6.2.16.7. Service Notice Message Object

The Service Notice Message Object extends the Base Message Object. It is not linked to a specific object or process, so `objectType`, `objectId` and `process` are normally absent.

Read schema:

```
{
  "$defs": {
    "serviceNoticeMessage.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "serviceNoticeMessage",
"readOnly": true },
        "objectType": { "type": "string", "readOnly": true },
        "objectId": { "type": "string", "readOnly": true },
        "message": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "message"]
    }
  }
}
```

6.3. Process Object Schemas

6.3.1. Base Process Object

The Base Process Object is extended by all Process Objects, which repeat its data elements as top-level properties.

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `processId` MUST be unique within the scope of the owning resource object.

Read schema:

```
{
  "$defs": {
    "baseProcess.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "baseProcess", "readOnly":
true },
        "processId": { "type": "string", "readOnly": true }
      },
      "required": ["@type"]
    }
  }
}
```

6.3.2. Create Process Object

The Create Process Object is extended by object-specific create processes and is submitted only within the `processes` data element of the owning resource object's create request.

Create request schema (create-only and read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/createProcess.create",
  "$defs": {
    "createProcess.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "createProcess" }
      },
      "required": ["@type"]
    }
  }
}
```

Read response schema (read-only properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/createProcess.read",
  "$defs": {
    "createProcess.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "createProcess", "readOnly":
true },
        "processId": { "type": "string", "readOnly": true }
      },
      "required": ["@type"]
    }
  }
}
```

Reference schema (identifier only):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/createProcess.reference",
  "$defs": {
    "createProcess.reference": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "createProcess" },
        "processId": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "processId"]
    }
  }
}
```

6.3.3. Renew Process Object

Renew request schema (create-only properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/renewProcess.create",
  "$defs": {
    "renewProcess.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "renewProcess" },
        "expiryDate": { "type": "string", "format": "date-time", "writeOnly":
true },
        "renewalPeriod": { "$ref": "#/$defs/period.create", "writeOnly":
true }
      },
      "required": ["@type"]
    }
  }
}
```

Renew response schema (read-only properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/renewProcess.read",
  "$defs": {
    "renewProcess.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "renewProcess", "readOnly":
true },
        "processId": { "type": "string", "readOnly": true },
        "expiryDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "renewalPeriod": { "$ref": "#/$defs/period.read", "readOnly": true }
      },
      "required": ["@type"]
    }
  }
}
```

Reference schema (identifier only):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/renewProcess.reference",
  "$defs": {
    "renewProcess.reference": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "renewProcess" },
        "processId": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "processId"]
    }
  }
}
```

6.3.4. Transfer Process Object

6.3.4.1. Create

Create request schema (create-only and read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/transferProcess.create",
  "$defs": {
    "transferProcess.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "transferProcess" },
        "transferDir": { "type": "string", "enum": ["pull", "push"],
"writeOnly": true },
        "gainingClientId": { "$ref": "#/$defs/clientIdentifier", "writeOnly":
true }
      },
      "required": ["@type"],
      "allOf": [
        {
          "if": {
            "properties": {
              "transferDir": { "const": "push" }
            },
            "required": ["transferDir"]
          },
          "then": { "required": ["gainingClientId"] }
        },
        {
          "if": {
            "properties": {
              "transferDir": { "const": "pull" }
            },
            "required": ["transferDir"]
          },
          "then": {
            "not": { "required": ["gainingClientId"] }
          }
        }
      ]
    }
  }
}
```

6.3.4.2. Read

Read response schema (read-write and read-only properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/transferProcess.read",
  "$defs": {
    "transferProcess.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "transferProcess", "readOnly":
true },
        "processId": { "type": "string", "readOnly": true },
        "transferDir": { "type": "string", "enum": ["pull", "push"],
```

```
"readOnly": true },
        "gainingClientId": { "$ref": "#/$defs/clientIdentifier", "readOnly":
true },
        "trStatus": {
          "type": "string",
          "enum": [
            "pending",
            "clientApproved",
            "clientCancelled",
            "clientRejected",
            "serverApproved",
            "serverCancelled"
          ],
          "readOnly": true
        },
        "reqClientId": { "$ref": "#/$defs/clientIdentifier", "readOnly":
true },
        "requestDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "actClientId": { "$ref": "#/$defs/clientIdentifier", "readOnly":
true },
        "actionDate": { "type": "string", "format": "date-time", "readOnly":
true }
      },
      "required": ["@type", "trStatus", "reqClientId", "requestDate",
"actClientId", "actionDate"],
      "allOf": [
        {
          "if": {
            "properties": {
              "transferDir": { "const": "push" }
            },
            "required": ["transferDir"]
          },
          "then": { "required": ["gainingClientId"] }
        },
        {
          "if": {
            "properties": {
              "transferDir": { "const": "pull" }
            },
            "required": ["transferDir"]
          },
          "then": {
            "not": { "required": ["gainingClientId"] }
          }
        }
      ]
    }
  }
}
```

6.3.4.3. Approve

The Approve operation MUST always have a request body, which may be an empty JSON object. The server can identify the transfer process to be approved based on the request URL and the authenticated client's permissions. There is no Approve request schema defined in this specification.

The Approve response schema MUST equal the read response schema.

6.3.4.4. Cancel

The Cancel operation, which is similar to a delete operation, has no request body, the server can identify the transfer process to be cancelled based on the request URL and the authenticated client's permissions. Because the Cancel operation has no request body, it is not possible to extend the request schema with additional properties.

The Cancel response schema MUST equal the read response schema or an empty response.

6.3.4.5. Reject

Reject request schema:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/transferProcess.transferReject",
  "$defs": {
    "transferProcess.transferReject": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "transferProcess" },
        "reason": { "type": "string" }
      },
      "required": ["@type"]
    }
  }
}
```

Reject response schema MUST equal the read response schema.

6.3.4.6. Reference

Reference schema (identifier only):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/transferProcess.reference",
  "$defs": {
    "transferProcess.reference": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "transferProcess" },
        "processId": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "processId"]
    }
  }
}
```

6.3.5. Restore Process Object

The Restore Process Object represents the current state of a restore request for an object that has entered the Redemption Grace Period (RGP). It is returned as a response for all restore operations.

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `requestDate` MUST NOT be present if no restore request has been submitted yet.
- `reportDate` MUST NOT be present if no restore report has been accepted yet.
- `reportDueDate` MUST NOT be present when `restoreStatus` is not `"pendingRestore"`.

Create request schema (create-only and read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/restoreProcess.create",
  "$defs": {
    "restoreProcess.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "restoreProcess" },
        "restoreReport": { "$ref": "#/$defs/restoreReport.create" }
      },
      "required": ["@type"]
    }
  }
}
```

Read response schema (read-write and read-only properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/restoreProcess.read",
  "$defs": {
    "restoreProcess.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "restoreProcess", "readOnly":
true },
        "processId": { "type": "string", "readOnly": true },
        "restoreStatus": {
          "type": "string",
          "enum": ["pendingRestore", "restored", "rgpPendingDelete"],
          "readOnly": true
        },
        "requestDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "reportDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "reportDueDate": { "type": "string", "format": "date-time",
"readOnly": true },
        "restoreReport": { "$ref": "#/$defs/restoreReport.read" }
      },
      "required": ["@type", "restoreStatus"]
    }
  }
}
```

The Update request applies only to servers that support or require submission of a restore report.

Update request schema (read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/restoreProcess.update",
  "$defs": {
    "restoreProcess.update": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "restoreProcess" },
        "restoreReport": { "$ref": "#/$defs/restoreReport.create" }
      },
      "required": ["@type", "restoreReport"]
    }
  }
}
```

Reference schema (identifier only):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/restoreProcess.reference",
  "$defs": {
    "restoreProcess.reference": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "restoreProcess" },
        "processId": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "processId"]
    }
  }
}
```

6.4. Domain Name Data Object

The Domain Name Data Object represents a domain name and its associated provisioning data.

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `name` MUST be a fully qualified domain name conforming to the syntax described in [RFC1035]. Servers MAY restrict allowable domain names to a specific namespace for which they are authoritative. The implicit trailing dot MUST NOT be included.
- `name` MUST use the ASCII Compatible Encoding (ACE) A-label form when the domain name is internationalised, as defined in [RFC5890].
- `uName`, when present, MUST be normalised to Unicode Normalization Form C (NFC) as defined in [UNICODE.NFC], and MUST convert to the exact value of `name` using the procedure described in Section 4.4.
- `lgr`, when present, MUST identify a valid Label Generation Ruleset (LGR) registered in the [IDN-Tables] registry, and MUST be present whenever `uName` is present.

6.4.1. Availability

The response of the Availability operation is the Domain Name Availability Object, see Section 6.2.15.2.

6.4.2. Create

Create request schema (create-only and read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/domainName.create",
  "$defs": {
    "domainName.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "domainName" },
        "name": { "type": "string", "format": "hostname", "writeOnly":
true },
        "uName": { "type": "string", "writeOnly": true },
        "lgr": { "type": "string", "writeOnly": true },
        "status": {
          "type": "array",
          "items": { "$ref": "#/$defs/domainStatus.create" }
        },
        "registrant": { "$ref": "#/$defs/contact.reference" },
        "contacts": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "label": { "type": "string" },
              "object": { "$ref": "#/$defs/contact.reference" }
            },
            "required": ["label", "object"]
          }
        },
        "nameservers": {
          "type": "array",
          "items": { "$ref": "#/$defs/host.reference" }
        },
        "dns": { "$ref": "#/$defs/dnsData.create" },
        "authInfo": { "$ref": "#/$defs/authInfo.create" },
        "processes": {
          "type": "object",
          "properties": {
            "@type": { "type": "string", "const": "processes" },
            "createProcess": {
              "type": "array",
              "items": { "$ref": "#/$defs/domainCreateProcess.create" }
            }
          },
          "required": ["@type"]
        }
      },
      "required": ["@type", "name"]
    }
  }
}
```

6.4.3. Create Process

The Domain Create Process Object extends the Create Process Object and is submitted only within the `processes` data element of the Domain Name create request.

Create request for Domain Create Process Object schema (create-only and read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/domainCreateProcess.create",
  "$defs": {
    "domainCreateProcess.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "domainCreateProcess" },
        "period": { "$ref": "#/$defs/period.create", "writeOnly": true }
      },
      "required": ["@type"]
    }
  }
}
```

Read response for Domain Create Process Object schema (read-write and read-only properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/domainCreateProcess.read",
  "$defs": {
    "domainCreateProcess.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "domainCreateProcess",
"readOnly": true },
        "processId": { "type": "string", "readOnly": true },
        "period": { "$ref": "#/$defs/period.read", "readOnly": true }
      },
      "required": ["@type"]
    }
  }
}
```

Reference for Domain Create Process Object schema (identifier only):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/domainCreateProcess.reference",
  "$defs": {
    "domainCreateProcess.reference": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "domainCreateProcess" },
        "processId": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "processId"]
    }
  }
}
```

6.4.4. Read

Read response schema (read-write and read-only properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/domainName.read",
  "$defs": {
    "domainName.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "domainName", "readOnly":
true },
        "name": { "type": "string", "format": "hostname", "readOnly": true },
        "uName": { "type": "string", "readOnly": true },
        "lgr": { "type": "string", "readOnly": true },
        "provMetadata": { "$ref": "#/$defs/provMetadata.read", "readOnly":
true },
        "status": {
          "type": "array",
          "items": { "$ref": "#/$defs/domainStatus.read" },
          "minItems": 1
        },
        "registrant": { "$ref": "#/$defs/contact.reference" },
        "contacts": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "label": { "type": "string" },
              "object": { "$ref": "#/$defs/contact.reference" }
            },
            "required": ["label", "object"]
          }
        },
        "nameservers": {
          "type": "array",
          "items": { "$ref": "#/$defs/host.reference" }
        },
        "dns": { "$ref": "#/$defs/dnsData.read" },
        "subordinateHosts": {
          "type": "array",
          "items": { "$ref": "#/$defs/host.reference" },
          "readOnly": true
        },
        "expiryDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "authInfo": { "$ref": "#/$defs/authInfo.read" },
        "processes": { "$ref": "#/$defs/processes.read", "readOnly": true }
      },
      "required": ["@type", "name", "provMetadata", "status"]
    }
  }
}
```

6.4.5. Update

Update request schema (read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/domainName.update",
  "$defs": {
    "domainName.update": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "domainName" },
        "status": {
          "type": "array",
          "items": { "$ref": "#/$defs/domainStatus.create" }
        },
        "registrant": { "$ref": "#/$defs/contact.reference" },
        "contacts": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "label": { "type": "string" },
              "object": { "$ref": "#/$defs/contact.reference" }
            },
            "required": ["label", "object"]
          }
        },
        "nameservers": {
          "type": "array",
          "items": { "$ref": "#/$defs/host.reference" }
        },
        "dns": { "$ref": "#/$defs/dnsData.create" },
        "authInfo": { "$ref": "#/$defs/authInfo.create" }
      },
      "required": ["@type"]
    }
  }
}
```

6.4.6. Transfer

Create request for Domain Transfer Process Object schema (create-only and read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/domainTransferProcess.create",
  "$defs": {
    "domainTransferProcess.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "domainTransferProcess" },
        "transferDir": { "type": "string", "enum": ["pull", "push"],
"writeOnly": true },
        "gainingClientId": { "$ref": "#/$defs/clientIdentifier", "writeOnly":
true },
        "transferPeriod": { "$ref": "#/$defs/period.create", "writeOnly":
true }
      },
      "required": ["@type"],
      "allOf": [
        {
          "if": {
            "properties": {
              "transferDir": { "const": "push" }
            },
            "required": ["transferDir"]
          },
          "then": { "required": ["gainingClientId"] }
        },
        {
          "if": {
            "properties": {
              "transferDir": { "const": "pull" }
            },
            "required": ["transferDir"]
          },
          "then": {
            "not": { "required": ["gainingClientId"] }
          }
        }
      ]
    }
  }
}
```

Read response for Domain Transfer Process Object schema (read-write and read-only properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/domainTransferProcess.read",
  "$defs": {
    "domainTransferProcess.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "domainTransferProcess",
"readOnly": true },
        "processId": { "type": "string", "readOnly": true },
```

```
        "transferDir": { "type": "string", "enum": ["pull", "push"],
"readOnly": true },
        "gainingClientId": { "$ref": "#/$defs/clientIdentifier", "readOnly":
true },
        "trStatus": {
          "type": "string",
          "enum": [
            "pending",
            "clientApproved",
            "clientCancelled",
            "clientRejected",
            "serverApproved",
            "serverCancelled"
          ],
          "readOnly": true
        },
        "reqClientId": { "$ref": "#/$defs/clientIdentifier", "readOnly":
true },
        "requestDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "actClientId": { "$ref": "#/$defs/clientIdentifier", "readOnly":
true },
        "actionDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "expiryDate": { "type": "string", "format": "date-time", "readOnly":
true },
        "transferPeriod": { "$ref": "#/$defs/period.read", "readOnly": true }
      },
      "required": ["@type", "trStatus", "reqClientId", "requestDate",
"actClientId", "actionDate"],
      "allOf": [
        {
          "if": {
            "properties": {
              "transferDir": { "const": "push" }
            },
            "required": ["transferDir"]
          },
          "then": { "required": ["gainingClientId"] }
        },
        {
          "if": {
            "properties": {
              "transferDir": { "const": "pull" }
            },
            "required": ["transferDir"]
          },
          "then": {
            "not": { "required": ["gainingClientId"] }
          }
        }
      ]
    }
  }
}
```

Both Approve and Delete (Cancel) operations have no request body, the server can identify the transfer process to be approved or cancelled based on the request URL and the authenticated client's permissions.

Approve and Delete response schemas are the `read` schema of `domainTransferProcess`.

Reject request Domain Object schema:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/domainTransferProcess.transferReject",
  "$defs": {
    "domainTransferProcess.transferReject": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "domainTransferProcess" },
        "reason": { "type": "string" }
      },
      "required": ["@type"]
    }
  }
}
```

Reject response schema is the `read` schema of `domainTransferProcess`.

Reference for Domain Transfer Process Object schema (identifier only):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/domainTransferProcess.reference",
  "$defs": {
    "domainTransferProcess.reference": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "domainTransferProcess" },
        "processId": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "processId"]
    }
  }
}
```

6.4.7. Reference

Reference schema (identifier only):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/domainName.reference",
  "$defs": {
    "domainName.reference": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "domainName" },
        "name": { "type": "string", "format": "hostname" }
      },
      "required": ["@type", "name"]
    }
  }
}
```

6.5. Contact Data Object

This document uses version 2.0 of JSContact [RFC9982] for the JSON representation of Contact Data Object contact information. The contact's name, postal address, phone numbers, email addresses, and other contact details are encapsulated in a JSContact `Card` object embedded in the `contactInfo` property.

6.5.1. JSContact Profile for RPP

Since JSContact is a general-purpose representation of contact data, this document defines a restricted usage profile for use within RPP, see [TODO: ref to RPP JSContact profile here: https://github.com/SIDN/ietf-rpp-jscontact-profile].

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `card.name.full` MUST be provided in the EPP Compatibility Profile.
- `card.addresses.addr` MUST be provided in the EPP Compatibility Profile, containing at least `countryCode` and a `"locality"` kind component.
- `card.emails.email.address` MUST be provided in the EPP Compatibility Profile.
- `card.addresses[*].countryCode` MUST be a valid two-character ISO 3166-1 [ISO3166-1] alpha-2 code when present.

6.5.2. Availability

The response of the Availability operation is the Contact Availability Object, see Section 6.2.15.3.

6.5.3. Create

Create request schema (create-only and read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/contact.create",
  "$defs": {
    "contact.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "contact" },
        "id": { "$ref": "#/$defs/identifier", "writeOnly": true },
        "status": {
          "type": "array",
          "items": { "$ref": "#/$defs/contactStatus.create" }
        },
        "contactInfo": { "$ref": "#/$defs/Card" },
        "authInfo": { "$ref": "#/$defs/authInfo.create" },
        "disclose": { "$ref": "#/$defs/disclose.read" }
      },
      "required": ["@type", "id", "contactInfo"]
    }
  }
}
```

6.5.4. Read

Read response schema (read-write and read-only properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/contact.read",
  "$defs": {
    "contact.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "contact", "readOnly": true },
        "id": { "$ref": "#/$defs/identifier", "readOnly": true },
        "provMetadata": { "$ref": "#/$defs/provMetadata.read", "readOnly":
true },
        "status": {
          "type": "array",
          "items": { "$ref": "#/$defs/contactStatus.read" },
          "minItems": 1
        },
        "contactInfo": { "$ref": "#/$defs/Card" },
        "authInfo": { "$ref": "#/$defs/authInfo.read" },
        "disclose": { "$ref": "#/$defs/disclose.read" },
        "processes": { "$ref": "#/$defs/processes.read", "readOnly": true }
      },
      "required": ["@type", "id", "provMetadata", "status", "contactInfo"]
    }
  }
}
```

6.5.5. Update

Update request schema (read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/contact.update",
  "$defs": {
    "contact.update": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "contact" },
        "status": {
          "type": "array",
          "items": { "$ref": "#/$defs/contactStatus.create" }
        },
        "contactInfo": { "$ref": "#/$defs/Card" },
        "authInfo": { "$ref": "#/$defs/authInfo.create" },
        "disclose": { "$ref": "#/$defs/disclose.read" }
      },
      "required": ["@type", "contactInfo"]
    }
  }
}
```

6.5.6. Reference

Reference schema (identifier only):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/contact.reference",
  "$defs": {
    "contact.reference": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "contact" },
        "id": { "$ref": "#/$defs/identifier" }
      },
      "required": ["@type", "id"]
    }
  }
}
```

6.6. Host Data Object

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `hostName` MUST be a syntactically valid fully qualified host name.
- If the host name is subordinate to a domain for which the server is authoritative, the superordinate domain MUST already exist in the server.

6.6.1. Availability

The response of the Availability operation is the Host Availability Object, see Section 6.2.15.4.

6.6.2. Create

Create request schema (create-only and read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/host.create",
  "$defs": {
    "host.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "host" },
        "hostName": { "type": "string", "format": "hostname" },
        "status": {
          "type": "array",
          "items": { "$ref": "#/$defs/hostStatus.create" }
        },
        "dns": { "$ref": "#/$defs/dnsData.create" }
      },
      "required": ["@type", "hostName"]
    }
  }
}
```

6.6.3. Read

Read response schema (read-write and read-only properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/host.read",
  "$defs": {
    "host.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "host", "readOnly": true },
        "hostName": { "type": "string", "format": "hostname" },
        "provMetadata": { "$ref": "#/$defs/provMetadata.read", "readOnly":
true },
        "status": {
          "type": "array",
          "items": { "$ref": "#/$defs/hostStatus.read" },
          "minItems": 1
        },
        "dns": { "$ref": "#/$defs/dnsData.read" },
        "processes": { "$ref": "#/$defs/processes.read", "readOnly": true }
      },
      "required": ["@type", "hostName", "provMetadata", "status"]
    }
  }
}
```

6.6.4. Update

Update request schema (read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/host.update",
  "$defs": {
    "host.update": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "host" },
        "hostName": { "type": "string", "format": "hostname" },
        "status": {
          "type": "array",
          "items": { "$ref": "#/$defs/hostStatus.create" }
        },
        "dns": { "$ref": "#/$defs/dnsData.create" }
      },
      "required": ["@type", "hostName"]
    }
  }
}
```

6.6.5. Reference

Reference schema (identifier only):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/host.reference",
  "$defs": {
    "host.reference": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "host" },
        "hostName": { "type": "string", "format": "hostname" }
      },
      "required": ["@type", "hostName"]
    }
  }
}
```

6.7. Organisation Data Object

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `id` MUST be a server-unique identifier conforming to the Identifier syntax.
- `status` MUST always contain at least one value. `pendingCreate`, `ok`, `hold`, and `terminated` are mutually exclusive. `ok` MAY only be combined with `linked`.

- `parent`, if present, MUST reference a known Organisation Object. Circular parent references MUST be rejected.
- `contacts` keys MUST be contact type values registered in the IANA "EPP Organisation Contact Types" registry ([RFC8543]).

6.7.1. Create

Create request schema (create-only and read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/organisation.create",
  "$defs": {
    "organisation.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "organisation" },
        "id": { "$ref": "#/$defs/identifier", "writeOnly": true },
        "status": {
          "type": "array",
          "items": { "$ref": "#/$defs/organisationStatus.create" }
        },
        "roles": {
          "type": "object",
          "additionalProperties": { "$ref": "#/$defs/
organisationRole.create" },
          "minProperties": 1
        },
        "parent": { "$ref": "#/$defs/organisation.reference" },
        "contactInfo": { "$ref": "#/$defs/Card" },
        "contacts": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "label": { "type": "string" },
              "object": { "$ref": "#/$defs/contact.reference" }
            },
            "required": ["label", "object"]
          }
        },
        "users": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "label": { "type": "string" },
              "object": { "$ref": "#/$defs/user.reference" }
            },
            "required": ["label", "object"]
          }
        }
      },
      "required": ["@type", "id", "roles"]
    }
  }
}
```

6.7.2. Read

Read response schema (read-write and read-only properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/organisation.read",
```

```
  "$defs": {
    "organisation.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "organisation", "readOnly":
true },
        "id": { "$ref": "#/$defs/identifier", "readOnly": true },
        "provMetadata": { "$ref": "#/$defs/provMetadata.read", "readOnly":
true },
        "status": {
          "type": "array",
          "items": { "$ref": "#/$defs/organisationStatus.read" },
          "minItems": 1
        },
        "roles": {
          "type": "object",
          "additionalProperties": { "$ref": "#/$defs/
organisationRole.read" },
          "minProperties": 1
        },
        "parent": { "$ref": "#/$defs/organisation.reference" },
        "contactInfo": { "$ref": "#/$defs/Card" },
        "contacts": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "label": { "type": "string" },
              "object": { "$ref": "#/$defs/contact.reference" }
            },
            "required": ["label", "object"]
          }
        },
        "users": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "label": { "type": "string" },
              "object": { "$ref": "#/$defs/user.reference" }
            },
            "required": ["label", "object"]
          }
        },
        "processes": { "$ref": "#/$defs/processes.read", "readOnly": true }
      },
      "required": ["@type", "id", "provMetadata", "status", "roles"]
    }
  }
}
```

6.7.3. Update

Update request schema (read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/organisation.update",
  "$defs": {
    "organisation.update": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "organisation" },
        "status": {
          "type": "array",
          "items": { "$ref": "#/$defs/organisationStatus.create" }
        },
        "roles": {
          "type": "object",
          "additionalProperties": { "$ref": "#/$defs/
organisationRole.create" },
          "minProperties": 1
        },
        "parent": { "$ref": "#/$defs/organisation.reference" },
        "contactInfo": { "$ref": "#/$defs/Card" },
        "contacts": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "label": { "type": "string" },
              "object": { "$ref": "#/$defs/contact.reference" }
            },
            "required": ["label", "object"]
          }
        },
        "users": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "label": { "type": "string" },
              "object": { "$ref": "#/$defs/user.reference" }
            },
            "required": ["label", "object"]
          }
        }
      },
      "required": ["@type", "roles"]
    }
  }
}
```

6.7.4. Reference

Reference schema (identifier only):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/organisation.reference",
  "$defs": {
    "organisation.reference": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "organisation" },
        "id": { "$ref": "#/$defs/identifier" }
      },
      "required": ["@type", "id"]
    }
  }
}
```

6.8. Message Data Object

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `data` is polymorphic: it MUST contain an instance of a message object that directly or indirectly extends the Base Message Object. The concrete message object is identified by the `@type` of the value used for the `data` property.

6.8.1. Create

Create request schema. Only the server is authorised to create Message Data Objects, so the representation also carries the server-managed data elements `id`, `creationDate` and `status`.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/message.create",
  "$defs": {
    "message.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "message" },
        "data": {
          "oneOf": [
            { "$ref": "#/$defs/reviewResponseMessage.read" },
            { "$ref": "#/$defs/transferRequestMessage.read" },
            { "$ref": "#/$defs/transferOutcomeMessage.read" },
            { "$ref": "#/$defs/expirationDeletionMessage.read" },
            { "$ref": "#/$defs/autoRenewalMessage.read" },
            { "$ref": "#/$defs/serviceNoticeMessage.read" }
          ],
          "writeOnly": true
        }
      },
      "required": ["@type", "data"]
    }
  }
}
```

6.8.2. Read

Read response schema (read-only properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/message.read",
  "$defs": {
    "message.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "message", "readOnly": true },
        "id": { "$ref": "#/$defs/identifier", "readOnly": true },
        "creationDate": { "type": "string", "format": "date-time",
"readOnly": true },
        "status": { "type": "string", "enum": ["queued", "delivered",
"removed"], "readOnly": true },
        "data": {
          "oneOf": [
            { "$ref": "#/$defs/reviewResponseMessage.read" },
            { "$ref": "#/$defs/transferRequestMessage.read" },
            { "$ref": "#/$defs/transferOutcomeMessage.read" },
            { "$ref": "#/$defs/expirationDeletionMessage.read" },
            { "$ref": "#/$defs/autoRenewalMessage.read" },
            { "$ref": "#/$defs/serviceNoticeMessage.read" }
          ],
          "readOnly": true
        }
      },
      "required": ["@type", "id", "creationDate", "status", "data"]
    }
  }
}
```

6.8.3. Query

Query response schema (array of message read representations):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/message.query",
  "$defs": {
    "message.query": {
      "type": "array",
      "items": { "$ref": "#/$defs/message.read" }
    }
  }
}
```

6.8.4. Reference

Reference schema (identifier only):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/message.reference",
  "$defs": {
    "message.reference": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "message" },
        "id": { "$ref": "#/$defs/identifier", "readOnly": true }
      },
      "required": ["@type", "id"]
    }
  }
}
```

6.9. User Data Object

The User Data Object represents a user linked to an Organisation Object. The lifecycle of a User Object is bound to its owning Organisation Object.

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations: - The `"status"` array MUST contain exactly one of the labels `"active"`, `"pending"`, `"suspended"`, or `"deactivated"`.

6.9.1. Create

Create request schema (create-only and read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/user.create",
  "$defs": {
    "user.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "user" },
        "id": { "$ref": "#/$defs/identifier", "writeOnly": true },
        "details": { "$ref": "#/$defs/Card" },
        "description": { "type": "string" },
        "status": { "$ref": "#/$defs/userStatus.create" }
      },
      "required": ["@type", "id", "details"]
    }
  }
}
```

6.9.2. Read

Read response schema (read-write and read-only properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/user.read",
  "$defs": {
    "user.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "user", "readOnly": true },
        "id": { "$ref": "#/$defs/identifier", "readOnly": true },
        "details": { "$ref": "#/$defs/Card" },
        "description": { "type": "string" },
        "status": { "$ref": "#/$defs/userStatus.read" },
        "organisation": { "$ref": "#/$defs/organisation.reference",
"readOnly": true },
        "processes": { "$ref": "#/$defs/processes.read", "readOnly": true }
      },
      "required": ["@type", "id", "details", "status", "organisation"]
    }
  }
}
```

6.9.3. Update

Update request schema (read-write properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/user.update",
  "$defs": {
    "user.update": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "user" },
        "details": { "$ref": "#/$defs/Card" },
        "description": { "type": "string" },
        "status": { "$ref": "#/$defs/userStatus.create" }
      },
      "required": ["@type", "details"]
    }
  }
}
```

6.9.4. Reference

Reference schema (identifier only):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/user.reference",
  "$defs": {
    "user.reference": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "user" },
        "id": { "$ref": "#/$defs/identifier" }
      },
      "required": ["@type", "id"]
    }
  }
}
```

6.10. Discovery Data Object

The Discovery Data Object describes the capabilities of an RPP server. The server provides a single Discovery Data Object, which has no Unique Identifier. All data elements are read-only, and the only supported operation is Read.

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `datetime` MUST be expressed in UTC using the `Z` suffix and MUST NOT use a numeric UTC offset.
- Each `object` value in `links` MUST be an absolute URL using the HTTPS scheme, and the document MUST be accessible to the client without authentication.
- Each `label` in `links` MUST be a link relation type registered in the IANA "Link Relation Types" registry.
- Each value in `objects` MUST be a valid object identifier as defined in the RPP Data Object Registry.

6.10.1. Read

Read response schema (read-only properties).

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/discovery.read",
  "$defs": {
    "discovery.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "discovery", "readOnly":
true },
        "versions": {
          "type": "array",
          "items": { "$ref": "#/$defs/discoveryVersion.read" },
          "minItems": 1,
          "readOnly": true
        },
        "environment": { "$ref": "#/$defs/discoveryEnvironment.read",
```

```
"readOnly": true },
        "tlds": {
          "type": "array",
          "items": { "type": "string" },
          "minItems": 1,
          "readOnly": true
        },
        "extensions": {
          "type": "array",
          "items": { "$ref": "#/$defs/extension.read" },
          "readOnly": true
        },
        "profiles": {
          "type": "array",
          "items": { "type": "string" },
          "readOnly": true
        },
        "objects": {
          "type": "array",
          "items": { "type": "string" },
          "minItems": 1,
          "readOnly": true
        },
        "datetime": { "type": "string", "format": "date-time", "readOnly":
true },
        "links": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "label": { "type": "string" },
              "object": { "type": "string" }
            },
            "required": ["label", "object"]
          },
          "readOnly": true
        },
        "idnLgr": {
          "type": "array",
          "items": { "$ref": "#/$defs/discoveryIdnLgr.read" },
          "readOnly": true
        },
        "notices": {
          "type": "array",
          "items": { "$ref": "#/$defs/discoveryNotice.read" },
          "readOnly": true
        }
      },
      "required": ["@type", "versions", "environment", "tlds", "objects",
"datetime"]
    }
  }
}
```

6.11. Profile Data Object

The Profile Data Object represents a named set of protocol features and versions published by the server. All data elements are read-only, and the only supported operation is Read.

The following constraints cannot be expressed in JSON Schema and MUST be enforced by implementations:

- `name` of a standard profile MUST use the RPP URN namespace, for example `urn:ietf:params:rpp:profile:{profile_name}`. The `name` of a private profile MUST be globally unique, the use of reverse domain name notation is RECOMMENDED.
- A profile MUST NOT list the same extension name more than once, and the server MUST support all extensions listed in the profile.
- The profile referenced by `baseProfile` MUST NOT itself have a `baseProfile`, and a profile MUST NOT be its own base profile.
- Only one profile MUST have `defaultProfile` set to `true`.

6.11.1. Read

Read response schema (read-only properties):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/profile.read",
  "$defs": {
    "profile.read": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "profile", "readOnly": true },
        "name": { "type": "string", "readOnly": true },
        "description": { "type": "string", "readOnly": true },
        "version": { "type": "string", "readOnly": true },
        "rppVersion": { "type": "string", "readOnly": true },
        "objects": {
          "type": "array",
          "items": { "type": "string" },
          "minItems": 1,
          "readOnly": true
        },
        "extensions": {
          "type": "array",
          "items": { "$ref": "#/$defs/extension.read" },
          "readOnly": true
        },
        "baseProfile": { "$ref": "#/$defs/profile.reference", "readOnly":
true },
        "defaultProfile": { "type": "boolean", "readOnly": true }
      },
      "required": [
        "@type",
        "name",
        "description",
        "version",
        "rppVersion",
        "objects",
        "defaultProfile"
      ]
    }
  }
}
```

6.11.2. Reference

Reference schema (identifier only):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/profile.reference",
  "$defs": {
    "profile.reference": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "profile" },
        "name": { "type": "string", "readOnly": true }
      },
      "required": ["@type", "name"]
    }
  }
}
```

7. Extension Framework

Extensions MUST employ an additive schema composition method, allowing new optional or required properties to be added to an existing base object schema. New properties MUST not conflict with existing definitions and MUST NOT redefine existing properties from the base schema.

> TODO: the rule above needs revision - extending enum list won't be possible this way

To support multiple extensions, a mechanism for mapping extensions to their respective base objects is defined using the `rpp:extends` property within the extension schema.

Extensions SHOULD be independent of one another, meaning that an extension should not rely on the presence of another extension to function correctly. This ensures modularity and reduces the risk of conflicts between extensions.

An extension schema MUST define a unique identifier using a top-level `$id` to ensure that references to the schema are unambiguous and correctly resolved. For IETF standardised extensions, using a stable and unique URI as the `$id` is RECOMMENDED.

An IANA registry for RPP JSON extensions is requested in this document, standardised JSON extensions SHALL be registered in this registry. Ensuring that each extension has a unique and stable identifier that can be referenced reliably across different implementations. This also promotes the reuse of common extensions and reduces duplication of effort.

It is RECOMMENDED to use the prefix "ext." for every extension sub-schema defined, for example `#/$defs/ext.domainFoo`.

TODO what objects are eligible for extensions?

7.1. Extension Mapping

Each extension MUST define a mapping to every base object it applies to, this is done by using a special `rpp:extends` property within the extension schema. For example, this mapping where the base object `https://rpp.example/rpp/schema.json#/$defs/domainName.create` is extended by the local definition `#/$defs/ext.domainFoo`:

```
{
  "rpp:extends": {
    "https://rpp.example/rpp/schema.json#/$defs/domainName.create":
      "#/$defs/ext.domainFoo"
  }
}
```

7.2. Effective Schema

Extensions are additive and independent, the effective schema for a base object MUST account for all applicable extensions to ensure that a validator can determine whether an object fully conforms to the combined set of constraints. When one or more extensions are applied to a base object, the effective schema for that object is obtained by composing the base object's schema with the schemas contributed by each extension using the `allOf` keyword, the server uses the `rpp:extends` metadata property to resolve the mapping between the base object and its extensions correctly.

The resulting schema is a self-contained schema that includes everything from both the base specification and the extensions, allowing clients to validate objects against a single comprehensive schema.

The following is an example of an extension JSON Schema, which adds new properties to both the domain and contact base objects:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://rpp.example/schemas/extension1.json",

  "rpp:extends": {
    "https://rpp.example/rpp/schema.json#/$defs/domainName.create":
      "#/$defs/ext.domain",

    "https://rpp.example/rpp/schema.json#/$defs/contact.create":
      "#/$defs/ext.contact"
  },

  "$defs": {
    "ext.domain": {
      "type": "object",
      "properties": {
        "foo": { "type": "string" },
        "bar": { "type": "boolean" }
      },
      "required": ["foo"]
    },

    "ext.contact": {
      "type": "object",
      "properties": {
        "fooContact": { "type": "string" }
      }
    }
  }
}
```

The effective schema is a distinct schema resource from the base schema it composes and therefore MUST be assigned its own `$id`, different from the `$id` of the base schema and of any other effective schema, to avoid ambiguous or circular references. It is RECOMMENDED to use the prefix "effective." for every effective sub-schema defined, for example `#/$defs/effective.domainFoo`.

Domain Name effective schema, which includes the full schema of the base domain object along with any applicable extensions:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://rpp.example/rpp/effective/domain.json",

  "$ref": "#/$defs/effective.domainName.create",
  "$defs": {
    "effective.domainName.create": {
      "allOf": [
        {
          "$ref": "https://rpp.example/rpp/schema.json#/$defs/
domainName.create"
        },
        {
          "$ref": "#/$defs/ext.domain"
        }
      ]
    },

    "ext.domain": {
      "type": "object",
      "properties": {
        "foo": {
          "type": "string"
        }
      },
      "required": ["foo"]
    }
  }
}
```

8. Examples

This section provides examples that follow the JSON representation rules and JSON Schema definitions specified in the previous sections. The examples illustrate typical request and response messages for the RPP Data Objects defined in [I-D.ietf-rpp-data-objects].

8.1. Domain Name

8.1.1. Availability

Example domain availability response for a domain name that can be created:

```
{
    "@type": "domainNameAvailability",
    "available": true
}
```

Example domain availability response for a domain name that cannot be created:

```
{
    "@type": "domainNameAvailability",
    "available": false,
    "reason": "In use"
}
```

8.1.2. Create

Example domain create request:

```
{
    "@type": "domainName",
    "name": "example.example",
    "nameservers": [
        { "@type": "host", "hostName": "ns1.example.example" },
        { "@type": "host", "hostName": "ns2.example.example" }
    ],
    "registrant": { "@type": "contact", "id": "JD-1234" },
    "contacts": [
        { "label": "admin", "object": { "@type": "contact", "id":
"SH-8013" } },
        { "label": "tech", "object": { "@type": "contact", "id":
"SH-8013" } }
    ],
    "authInfo": {
        "@type": "authInfo",
        "method": "authinfo",
        "authdata": "2fooBAR"
    },
    "processes": {
        "@type": "processes",
        "createProcess": [
            {
                "@type": "domainCreateProcess",
                "period": {
                    "@type": "period",
                    "value": 2,
                    "unit": "y"
                }
            }
        ]
    }
}
```

Example domain create response from a server with RGP support:

```
{
    "@type": "domainName",
    "name": "example.example",
    "provMetadata": {
        "@type": "provMetadata",
        "repositoryId": "EXAMPLE1-REP",
        "spClientId": "ClientX",
        "crClientId": "ClientX",
        "crDate": "1999-04-03T22:00:00.0Z"
    },
    "status": [
        { "@type": "domainStatus", "label": "ok" },
        { "@type": "domainStatus", "label": "rgpAddPeriod" }
    ],
    "expiryDate": "2001-04-03T22:00:00.0Z"
}
```

8.1.3. Create Process Data

Example Create Process Object without object-specific create inputs, as returned in a read response:

```
{
    "@type": "createProcess",
    "processId": "CRT-1002"
}
```

Example Create Process Object for a two-year domain registration period, as embedded above in `processes.createProcess`:

```
{
    "@type": "domainCreateProcess",
    "period": {
        "@type": "period",
        "value": 2,
        "unit": "y"
    }
}
```

8.1.4. Read

Example domain read response:

```
{
    "@type": "domainName",
    "name": "example.example",
    "provMetadata": {
        "@type": "provMetadata",
        "repositoryId": "EXAMPLE1-REP",
        "spClientId": "ClientX",
        "crClientId": "ClientY",
        "crDate": "1999-04-03T22:00:00.0Z",
```

```
        "upClientId": "ClientX",
        "upDate": "1999-12-03T09:00:00.0Z",
        "trDate": "2000-04-08T09:00:00.0Z"
    },
    "status": [
        { "@type": "domainStatus", "label": "ok" }
    ],
    "registrant": { "@type": "contact", "id": "JD-1234" },
    "contacts": [
        { "label": "admin", "object": { "@type": "contact", "id":
"SH-8013" } },
        { "label": "tech", "object": { "@type": "contact", "id":
"SH-8013" } }
    ],
    "nameservers": [
        {
            "@type": "host",
            "hostName": "ns1.example.example"
        },
        {
            "@type": "host",
            "hostName": "ns2.example.example"
        }
    ],
    "subordinateHosts": [
        {
            "@type": "host",
            "hostName": "ns1.example.example"
        },
        {
            "@type": "host",
            "hostName": "ns2.example.example"
        }
    ],
    "expiryDate": "2005-04-03T22:00:00.0Z",
    "authInfo": {
        "@type": "authInfo",
        "method": "authinfo",
        "authdata": "2fooBAR"
    },
    "processes": {
        "@type": "processes",
        "createProcess": [
            { "@type": "domainCreateProcess", "processId": "CRT-1001" }
        ]
    }
}
```

Example Read response for an internationalised domain name, showing the ACE `name`, the corresponding `uName`, and the `lgr` identifying the Label Generation Ruleset (LGR) used for validation:

```
{
    "@type": "domainName",
    "name": "xn--bcher-kva.example",
    "uName": "bücher.example",
    "lgr": "latn-1.0",
    "provMetadata": {
        "@type": "provMetadata",
        "repositoryId": "BUCHER1-REP",
        "spClientId": "ClientX",
        "crClientId": "ClientX",
        "crDate": "1999-04-03T22:00:00.0Z"
    },
    "status": [
        { "@type": "domainStatus", "label": "ok" }
    ]
}
```

8.1.5. Update

Example domain update request (read-write properties):

```
{
    "@type": "domainName",
    "registrant": { "@type": "contact", "id": "SH-8013" },
    "authInfo": {
        "@type": "authInfo",
        "method": "authinfo",
        "authdata": "2BARfoo"
    }
}
```

Example domain update response:

```
{
    "@type": "domainName",
    "name": "example.example",
    "provMetadata": {
        "@type": "provMetadata",
        "repositoryId": "EXAMPLE1-REP",
        "spClientId": "ClientX",
        "crClientId": "ClientY",
        "crDate": "1999-04-03T22:00:00.0Z",
        "upClientId": "ClientX",
        "upDate": "2000-01-15T09:00:00.0Z"
    },
    "status": [
        { "@type": "domainStatus", "label": "ok" }
    ],
    "registrant": { "@type": "contact", "id": "SH-8013" }
}
```

8.1.6. Delete

The domain delete operation takes the domain name as the resource identifier in the request. No request body is required.

Example domain delete response (minimal, server may return full representation):

```
{
    "@type": "domainName",
    "name": "example.example",
    "provMetadata": {
        "@type": "provMetadata",
        "repositoryId": "EXAMPLE1-REP",
        "spClientId": "ClientX"
    },
    "status": [
        { "@type": "domainStatus", "label": "pendingDelete" }
    ]
}
```

8.1.7. Renew

The renew operation creates a Renew Process Object. The optional `expiryDate` in the request is the current expiry date, sent by the client for server-side validation to prevent duplicate renewals. The response returns a Renew Process Object containing the new expiry date.

Example domain renew request:

```
{
    "@type": "renewProcess",
    "expiryDate": "2005-04-03T22:00:00.0Z",
    "renewalPeriod": {
        "@type": "period",
        "value": 5,
        "unit": "y"
    }
}
```

Example domain renew response:

```
{
    "@type": "renewProcess",
    "expiryDate": "2010-04-03T22:00:00.0Z"
}
```

8.1.8. Transfer Request

Authorisation information for the transfer MUST be conveyed using the `RPP-Authorization` HTTP header (see Authorisation Information in Requests), not in the JSON request body.

Example domain transfer request (pull transfer):

```
{
    "@type": "domainTransferProcess",
    "transferDir": "pull",
    "transferPeriod": {
        "@type": "period",
        "value": 1,
        "unit": "y"
    }
}
```

Example domain transfer response (Transfer Process Object):

```
{
    "@type": "domainTransferProcess",
    "trStatus": "pending",
    "transferDir": "pull",
    "reqClientId": "ClientX",
    "requestDate": "2000-06-08T22:00:00.0Z",
    "actClientId": "ClientY",
    "actionDate": "2000-06-13T22:00:00.0Z",
    "expiryDate": "2002-09-08T22:00:00.0Z"
}
```

8.1.9. Transfer Query

Example domain transfer query response (Transfer Process Object):

```
{
    "@type": "domainTransferProcess",
    "trStatus": "pending",
    "transferDir": "pull",
    "reqClientId": "ClientX",
    "requestDate": "2000-06-06T22:00:00.0Z",
    "actClientId": "ClientY",
    "actionDate": "2000-06-11T22:00:00.0Z",
    "expiryDate": "2002-09-08T22:00:00.0Z"
}
```

8.1.10. Transfer Approve

The Transfer approve operation has a request body with an empty JSON object, the server can identify the transfer process to be approved based on the request URL and the authenticated client's permissions.

Example domain transfer approve request with empty JSON object:

```
{
  "@type": "domainTransferProcess"
}
```

The response is the Transfer Process Object, similar to the Create (Transfer Request) and Read (Transfer Query) responses.

8.1.11. Transfer Cancel

The Transfer cancel operation has an empty request body, the server can identify the transfer process to be cancelled based on the request URL and the authenticated client's permissions.

The response is the Transfer Process Object, similar to the Create (Transfer Request) and Read (Transfer Query) responses.

8.1.12. Transfer Reject

Example domain transfer reject request:

```
{
    "@type": "domainTransferProcess",
    "reason": "Client rejected the transfer request."
}
```

The transfer reject response is the Transfer Process Object, similar to the Create (Transfer Request) and Read (Transfer Query) responses.

8.1.13. Restore Request

Example domain restore request (without inline report; object transitions to `pendingRestore` state):

```
{
    "@type": "restoreProcess"
}
```

Example domain restore response (Restore Process Object, server requires a report):

```
{
    "@type": "restoreProcess",
    "restoreStatus": "pendingRestore",
    "requestDate": "2025-01-20T15:30:00.0Z",
    "reportDueDate": "2025-01-27T15:30:00.0Z"
}
```

Example domain restore request with inline restore report (single-step; object restored immediately):

```
{
    "@type": "restoreProcess",
    "restoreReport": {
        "@type": "restoreReport",
        "preData": "Domain example.example was registered on 2024-01-15 with
registrant JD-1234.",
        "postData": "Domain example.example is being restored with the same
registration data.",
        "deleteTime": "2025-01-10T12:00:00.0Z",
        "restoreTime": "2025-01-20T15:30:00.0Z",
        "restoreReason": "Domain deleted in error by client operator.",
        "statements": [
            "The information in this report is true to the best of my
knowledge.",
            "I have a valid reason for restoring this domain name."
        ]
    }
}
```

Example domain restore response with inline report (Restore Process Object, immediately restored):

```
{
    "@type": "restoreProcess",
    "restoreStatus": "restored",
    "requestDate": "2025-01-20T15:30:00.0Z",
    "reportDate": "2025-01-20T15:30:00.0Z"
}
```

8.1.14. Restore Report

Example domain restore report request:

```
{
    "@type": "restoreProcess",
    "restoreReport": {
        "@type": "restoreReport",
        "preData": "Domain example.example was registered on 2024-01-15 with
registrant JD-1234.",
        "postData": "Domain example.example is being restored with the same
registration data.",
        "deleteTime": "2025-01-10T12:00:00.0Z",
        "restoreTime": "2025-01-20T15:30:00.0Z",
        "restoreReason": "Domain deleted in error by client operator.",
        "statements": [
            "The information in this report is true to the best of my
knowledge.",
            "I have a valid reason for restoring this domain name."
        ]
    }
}
```

Example domain restore report response (Restore Process Object):

```
{
    "@type": "restoreProcess",
    "restoreStatus": "restored",
    "requestDate": "2025-01-20T15:30:00.0Z",
    "reportDate": "2025-01-22T09:15:00.0Z"
}
```

8.1.15. Read (Restore Query)

The Read (Restore Query) operation takes no request body (Parameters: None).

Example domain restore query response (Restore Process Object, object in `pendingRestore` state):

```
{
    "@type": "restoreProcess",
    "restoreStatus": "pendingRestore",
    "requestDate": "2025-01-20T15:30:00.0Z",
    "reportDueDate": "2025-01-27T15:30:00.0Z"
}
```

Example domain restore query response (Restore Process Object, object restored):

```
{
    "@type": "restoreProcess",
    "restoreStatus": "restored",
    "requestDate": "2025-01-20T15:30:00.0Z",
    "reportDate": "2025-01-22T09:15:00.0Z"
}
```

8.2. Contact

8.2.1. Availability

Example contact availability response for a contact that cannot be created:

```
{
    "@type": "contactAvailability",
    "available": false,
    "reason": "In use"
}
```

8.2.2. Create

Example contact create request:

```
{
    "@type": "contact",
    "id": "JD-1234",
    "contactInfo": {
        "@type": "Card",
        "version": "2.0",
        "kind": "individual",
        "name": {
            "full": "John Doe"
        },
        "organizations": {
            "org": { "name": "Example Inc." }
        },
        "addresses": {
            "addr": {
                "components": [
                    { "kind": "name",     "value": "123 Example Dr., Suite
100" },
                    { "kind": "locality", "value": "Dulles" },
                    { "kind": "region",   "value": "VA" },
                    { "kind": "postcode", "value": "20166-6503" }
                ],
                "countryCode": "US"
            }
        },
        "phones": {
            "voice": { "number": "tel:+1-703-555-5555" },
            "fax":   { "features": { "fax": true }, "number": "tel:
+1-703-555-5556" }
        },
        "emails": {
            "email": { "address": "jdoe@example.example" }
        }
    },
    "authInfo": {
        "@type": "authInfo",
        "method": "authinfo",
        "authdata": "2fooBAR"
    }
}
```

Example contact create response:

```
{
    "@type": "contact",
    "id": "JD-1234",
    "provMetadata": {
        "@type": "provMetadata",
        "repositoryId": "JD1234-REP",
        "spClientId": "ClientX",
        "crClientId": "ClientX",
        "crDate": "1999-04-03T22:00:00.0Z"
    },
    "status": [
        { "@type": "contactStatus", "label": "ok" }
    ],
    "contactInfo": {
        "@type": "Card",
        "version": "2.0",
        "kind": "individual",
        "name": {
            "full": "John Doe"
        },
        "organizations": {
            "org": { "name": "Example Inc." }
        },
        "addresses": {
            "addr": {
                "components": [
                    { "kind": "name",     "value": "123 Example Dr., Suite
100" },
                    { "kind": "locality", "value": "Dulles" },
                    { "kind": "region",   "value": "VA" },
                    { "kind": "postcode", "value": "20166-6503" }
                ],
                "countryCode": "US"
            }
        },
        "phones": {
            "voice": { "number": "tel:+1-703-555-5555" },
            "fax":   { "features": { "fax": true }, "number": "tel:
+1-703-555-5556" }
        },
        "emails": {
            "email": { "address": "jdoe@example.example" }
        }
    }
}
```

8.2.3. Read

Example contact read response:

```
{
    "@type": "contact",
    "id": "JD-1234",
    "provMetadata": {
        "@type": "provMetadata",
        "repositoryId": "JD1234-REP",
        "spClientId": "ClientX",
        "crClientId": "ClientX",
        "crDate": "1999-04-03T22:00:00.0Z",
        "upClientId": "ClientX",
        "upDate": "2000-01-15T09:00:00.0Z"
    },
    "status": [
        { "@type": "contactStatus", "label": "ok" }
    ],
    "contactInfo": {
        "@type": "Card",
        "version": "2.0",
        "kind": "individual",
        "name": {
            "full": "John Doe"
        },
        "organizations": {
            "org": { "name": "Example Inc." }
        },
        "addresses": {
            "addr": {
                "components": [
                    { "kind": "name",     "value": "123 Example Dr., Suite
100" },
                    { "kind": "locality", "value": "Dulles" },
                    { "kind": "region",   "value": "VA" },
                    { "kind": "postcode", "value": "20166-6503" }
                ],
                "countryCode": "US"
            }
        },
        "phones": {
            "voice": { "number": "tel:+1-703-555-5555" }
        },
        "emails": {
            "email": { "address": "jdoe@example.example" }
        }
    }
}
```

8.2.4. Update

Example contact update request:

```
{
    "@type": "contact",
    "contactInfo": {
        "@type": "Card",
        "version": "2.0",
        "addresses": {
            "addr": {
                "components": [
                    { "kind": "name",     "value": "456 New Street, Suite
200" },
                    { "kind": "locality", "value": "Reston" },
                    { "kind": "region",   "value": "VA" },
                    { "kind": "postcode", "value": "20190" }
                ],
                "countryCode": "US"
            }
        },
        "phones": {
            "voice": { "number": "tel:+1-703-555-5556" }
        },
        "emails": {
            "email": { "address": "jdoe-new@example.example" }
        }
    }
}
```

Example contact update response:

```
{
    "@type": "contact",
    "id": "JD-1234",
    "provMetadata": {
        "@type": "provMetadata",
        "repositoryId": "JD1234-REP",
        "spClientId": "ClientX",
        "crClientId": "ClientX",
        "crDate": "1999-04-03T22:00:00.0Z",
        "upClientId": "ClientX",
        "upDate": "2025-06-01T10:00:00.0Z"
    },
    "status": [
        { "@type": "contactStatus", "label": "ok" }
    ],
    "contactInfo": {
        "@type": "Card",
        "version": "2.0",
        "kind": "individual",
        "name": {
            "full": "John Doe"
        },
        "organizations": {
            "org": { "name": "Example Inc." }
        },
        "addresses": {
            "addr": {
                "components": [
                    { "kind": "name",     "value": "456 New Street, Suite
200" },
                    { "kind": "locality", "value": "Reston" },
                    { "kind": "region",   "value": "VA" },
                    { "kind": "postcode", "value": "20190" }
                ],
                "countryCode": "US"
            }
        },
        "phones": {
            "voice": { "number": "tel:+1-703-555-5556" }
        },
        "emails": {
            "email": { "address": "jdoe-new@example.example" }
        }
    }
}
```

8.2.5. Delete

The contact delete operation takes the contact identifier as the resource identifier. No request body is required.

8.2.6. Transfer Request

Authorisation information for the transfer MUST be conveyed using the `RPP-Authorization` HTTP header (see Authorisation Information in Requests), not in the JSON request body.

Example contact transfer request (pull transfer):

```
{
    "@type": "transferProcess",
    "transferDir": "pull"
}
```

Example contact transfer response (Transfer Process Object):

```
{
    "@type": "transferProcess",
    "trStatus": "pending",
    "transferDir": "pull",
    "reqClientId": "ClientX",
    "requestDate": "2000-06-08T22:00:00.0Z",
    "actClientId": "ClientY",
    "actionDate": "2000-06-13T22:00:00.0Z"
}
```

8.2.7. Transfer Query

Example contact transfer query response (Transfer Process Object):

```
{
    "@type": "transferProcess",
    "trStatus": "pending",
    "transferDir": "pull",
    "reqClientId": "ClientX",
    "requestDate": "2000-06-06T22:00:00.0Z",
    "actClientId": "ClientY",
    "actionDate": "2000-06-11T22:00:00.0Z"
}
```

8.2.8. Transfer Cancel / Approve

Transfer cancel and approve responses return the Transfer Process Object. The response structure is the same as the Transfer Query response above. The `trStatus` value reflects the outcome of the operation (e.g. `"clientCancelled"`, `"clientRejected"`, or `"clientApproved"`).

8.2.9. Transfer Reject

Example contact transfer reject request:

```
{
    "@type": "transferProcess",
    "reason": "Client rejected the transfer request."
}
```

The transfer reject response is the Transfer Process Object, similar to the Create (Transfer Request) and Read (Transfer Query) responses.

8.3. Host

8.3.1. Availability

Example host availability response for a host that can be created:

```
{
    "@type": "hostAvailability",
    "available": true
}
```

8.3.2. Create

Example host create request:

```
{
    "@type": "host",
    "hostName": "ns1.example.example",
    "dns": {
        "@type": "dnsData",
        "records": [
            {
                "@type": "dnsRecord",
                "name": "@",
                "type": "ns",
                "rdata": { "nsdname": "ns1.example.example." }
            },
            {
                "@type": "dnsRecord",
                "name": "ns1.example.example.",
                "type": "a",
                "rdata": { "address": "192.0.2.1" }
            },
            {
                "@type": "dnsRecord",
                "name": "ns1.example.example.",
                "type": "aaaa",
                "rdata": { "address": "2001:db8::1" }
            }
        ],
        "controls": {
            "@type": "dnsControls",
            "ttl": { "a": 3600, "aaaa": 3600 }
        }
    }
}
```

Example host create response:

```
{
    "@type": "host",
    "hostName": "ns1.example.example",
    "provMetadata": {
        "@type": "provMetadata",
        "repositoryId": "NS1-REP",
        "spClientId": "ClientX",
        "crClientId": "ClientX",
        "crDate": "1999-04-03T22:00:00.0Z"
    },
    "status": [
        { "@type": "hostStatus", "label": "ok" }
    ],
    "dns": {
        "@type": "dnsData",
        "records": [
            {
                "@type": "dnsRecord",
                "name": "@",
                "type": "ns",
                "rdata": { "nsdname": "ns1.example.example." }
            },
            {
                "@type": "dnsRecord",
                "name": "ns1.example.example.",
                "type": "a",
                "rdata": { "address": "192.0.2.1" }
            },
            {
                "@type": "dnsRecord",
                "name": "ns1.example.example.",
                "type": "aaaa",
                "rdata": { "address": "2001:db8::1" }
            }
        ],
        "controls": {
            "@type": "dnsControls",
            "ttl": { "a": 3600, "aaaa": 3600 }
        }
    }
}
```

8.3.3. Read

Example host read response:

```
{
    "@type": "host",
    "hostName": "ns1.example.example",
    "provMetadata": {
        "@type": "provMetadata",
        "repositoryId": "NS1-REP",
        "spClientId": "ClientX",
        "crClientId": "ClientY",
        "crDate": "1999-04-03T22:00:00.0Z"
    },
    "status": [
        { "@type": "hostStatus", "label": "ok" }
    ],
    "dns": {
        "@type": "dnsData",
        "records": [
            {
                "@type": "dnsRecord",
                "name": "@",
                "type": "ns",
                "rdata": { "nsdname": "ns1.example.example." }
            },
            {
                "@type": "dnsRecord",
                "name": "ns1.example.example.",
                "type": "a",
                "rdata": { "address": "192.0.2.1" }
            }
        ]
    }
}
```

8.3.4. Update

Example host update request:

```
{
    "@type": "host",
    "hostName": "ns1.example.example",
    "dns": {
        "@type": "dnsData",
        "records": [
            {
                "@type": "dnsRecord",
                "name": "@",
                "type": "ns",
                "rdata": { "nsdname": "ns1.example.example." }
            },
            {
                "@type": "dnsRecord",
                "name": "ns1.example.example.",
                "type": "a",
                "rdata": { "address": "198.51.100.1" }
            }
        ]
    }
}
```

8.3.5. Delete

The host delete operation takes the host name as the resource identifier. No request body is required. The server SHOULD reject the request if the host object is associated with any domain name objects.

8.3.6. Restore Request

Example host restore request (without inline report; object transitions to `pendingRestore` state):

```
{
  "@type": "restoreProcess"
}
```

Example host restore request response (Restore Process Object, server requires a report):

```
{
    "@type": "restoreProcess",
    "restoreStatus": "pendingRestore",
    "requestDate": "2025-01-20T15:30:00.0Z",
    "reportDueDate": "2025-01-27T15:30:00.0Z"
}
```

Example host restore request with inline restore report (single-step; object restored immediately):

```
{
    "@type": "restoreProcess",
    "restoreReport": {
        "@type": "restoreReport",
        "preData": "Host ns1.example.example was registered on 2024-01-15 by
ClientX.",
        "postData": "Host ns1.example.example is being restored with the same
registration data.",
        "deleteTime": "2025-01-10T12:00:00.0Z",
        "restoreTime": "2025-01-20T15:30:00.0Z",
        "restoreReason": "Host deleted in error by client operator.",
        "statements": [
            "The information in this report is true to the best of my
knowledge.",
            "I have a valid reason for restoring this host object."
        ]
    }
}
```

Example host restore response with inline report (Restore Process Object, immediately restored):

```
{
    "@type": "restoreProcess",
    "restoreStatus": "restored",
    "requestDate": "2025-01-20T15:30:00.0Z",
    "reportDate": "2025-01-20T15:30:00.0Z"
}
```

8.3.7. Restore Report

Example host restore report request:

```
{
    "@type": "restoreProcess",
    "restoreReport": {
        "@type": "restoreReport",
        "preData": "Host ns1.example.example was registered on 2024-01-15 by
ClientX.",
        "postData": "Host ns1.example.example is being restored with the same
registration data.",
        "deleteTime": "2025-01-10T12:00:00.0Z",
        "restoreTime": "2025-01-20T15:30:00.0Z",
        "restoreReason": "Host deleted in error by client operator.",
        "statements": [
            "The information in this report is true to the best of my
knowledge.",
            "I have a valid reason for restoring this host object."
        ]
    }
}
```

Example host restore report response (Restore Process Object):

```
{
    "@type": "restoreProcess",
    "restoreStatus": "restored",
    "requestDate": "2025-01-20T15:30:00.0Z",
    "reportDate": "2025-01-22T09:15:00.0Z"
}
```

8.3.8. Read (Restore Query)

The Read (Restore Query) operation takes no request body (Parameters: None).

Example host restore query response (Restore Process Object, object in `pendingRestore` state):

```
{
    "@type": "restoreProcess",
    "restoreStatus": "pendingRestore",
    "requestDate": "2025-01-20T15:30:00.0Z",
    "reportDueDate": "2025-01-27T15:30:00.0Z"
}
```

Example host restore query response (Restore Process Object, object restored):

```
{
    "@type": "restoreProcess",
    "restoreStatus": "restored",
    "requestDate": "2025-01-20T15:30:00.0Z",
    "reportDate": "2025-01-22T09:15:00.0Z"
}
```

8.4. Organisation

8.4.1. Create

Example organisation create request:

```
{
    "@type": "organisation",
    "id": "ORG-12345",
    "roles": { "registrar":
        {
            "@type": "organisationRole",
            "roleId": "1234"
        }
    },
    "parent": { "@type": "organisation", "id": "ORG-9999" },
    "contactInfo": {
        "@type": "Card",
        "version": "2.0",
        "kind": "org",
        "name": {
            "full": "Example Registrar Inc."
        },
        "addresses": {
            "addr": {
                "components": [
                    { "kind": "name",     "value": "Meander 501" },
                    { "kind": "locality", "value": "Arnhem" },
                    { "kind": "region",   "value": "Gelderland" },
                    { "kind": "postcode", "value": "6825MD" }
                ],
                "countryCode": "NL"
            }
        },
        "emails": {
            "email": { "address": "registrar@example.example" }
        }
    },
    "contacts": [
        { "label": "admin", "object": { "@type": "contact", "id":
"CID-1001" } },
        { "label": "tech", "object": { "@type": "contact", "id":
"CID-1002" } }
    ]
}
```

Example organisation create response:

```
{
    "@type": "organisation",
    "id": "ORG-12345",
    "provMetadata": {
        "@type": "provMetadata",
        "repositoryId": "ORG12345-REP",
        "spClientId": "ClientX",
        "crClientId": "ClientX",
        "crDate": "2025-03-15T10:00:00.0Z"
    },
    "status": [
        { "@type": "organisationStatus", "label": "ok" }
    ],
```

```
    "roles": { "registrar":
        {
            "@type": "organisationRole",
            "status": [ { "@type": "organisationRoleStatus", "label":
"ok" } ],
            "roleId": "1234"
        }
    },
    "parent": { "@type": "organisation", "id": "ORG-9999" },
    "contactInfo": {
        "@type": "Card",
        "version": "2.0",
        "kind": "org",
        "name": {
            "full": "Example Registrar Inc."
        },
        "addresses": {
            "addr": {
               "components": [
                    { "kind": "name",     "value": "Meander 501" },
                    { "kind": "locality", "value": "Arnhem" },
                    { "kind": "region",   "value": "Gelderland" },
                    { "kind": "postcode", "value": "6825MD" }
                ],
                "countryCode": "NL"
            }
        },
        "emails": {
            "email": { "address": "registrar@example.example" }
        }
    },
    "contacts": [
        { "label": "admin", "object": { "@type": "contact", "id":
"CID-1001" } },
        { "label": "tech", "object": { "@type": "contact", "id":
"CID-1002" } }
    ],
    "users": [
        { "label": "admin", "object": { "@type": "user", "id": "UID-5001" } }
    ]
}
```

8.4.2. Read

Example organisation read response:

```
{
    "@type": "organisation",
    "id": "ORG-12345",
    "provMetadata": {
        "@type": "provMetadata",
        "repositoryId": "ORG12345-REP",
        "spClientId": "ClientX",
        "crClientId": "ClientX",
        "crDate": "2025-03-15T10:00:00.0Z",
        "upClientId": "ClientX",
        "upDate": "2025-06-01T08:30:00.0Z"
```

```
    },
    "status": [
        { "@type": "organisationStatus", "label": "linked" }
    ],
    "roles": { "registrar":
        {
            "@type": "organisationRole",
            "status": [ { "@type": "organisationRoleStatus", "label":
"ok" } ],
            "roleId": "1234"
        }
    },
    "parent": { "@type": "organisation", "id": "ORG-9999" },
    "contactInfo": {
        "@type": "Card",
        "version": "2.0",
        "kind": "org",
        "name": {
            "full": "Example Registrar Inc."
        },
        "addresses": {
            "addr": {
                "components": [
                    { "kind": "name",     "value": "Meander 501" },
                    { "kind": "locality", "value": "Arnhem" },
                    { "kind": "region",   "value": "Gelderland" },
                    { "kind": "postcode", "value": "6825MD" }
                ],
                "countryCode": "NL"
            }
        },
        "emails": {
            "email": { "address": "registrar@example.example" }
        }
    },
    "contacts": [
        { "label": "admin", "object": { "@type": "contact", "id":
"CID-1001" } },
        { "label": "tech", "object": { "@type": "contact", "id":
"CID-1002" } }
    ],
    "users": [
        { "label": "admin", "object": { "@type": "user", "id": "UID-5001" } }
    ]
}
```

8.4.3. Update

Example organisation update request:

```
{
    "@type": "organisation",
    "roles": { "registrar":
        {
            "@type": "organisationRole",
            "status": [ { "@type": "organisationRoleStatus", "label":
"clientLinkProhibited" } ],
            "roleId": "1234"
        }
    },
    "status": [
        { "@type": "organisationStatus", "label": "clientLinkProhibited" }
    ],
    "contacts": [
        { "label": "admin", "object": { "@type": "contact", "id":
"CID-2001" } },
        { "label": "tech", "object": { "@type": "contact", "id":
"CID-2002" } }
    ]
}
```

8.4.4. Delete

The organisation delete operation takes the organisation identifier as the resource identifier. No request body is required. The server MUST reject the request if the organisation object is associated with any other objects.

8.4.5. Reference

Example organisation reference (used when referencing an organisation from another object):

```
{
    "@type": "organisation",
    "id": "ORG-12345"
}
```

8.5. User

8.5.1. Create

Example user create request:

```
{
    "@type": "user",
    "id": "UID-5001",
    "details": {
        "@type": "Card",
        "version": "2.0",
        "kind": "individual",
        "name": { "full": "John Doe" },
        "emails": {
            "email": { "address": "john.doe@example.example" }
        }
    }
}
```

Example user create response:

```
{
    "@type": "user",
    "id": "UID-5001",
    "details": {
        "@type": "Card",
        "version": "2.0",
        "kind": "individual",
        "name": { "full": "John Doe" },
        "emails": {
            "email": { "address": "john.doe@example.example" }
        }
    },
    "status": { "@type": "userStatus", "label": "pending" },
    "organisation": { "@type": "organisation", "id": "ORG-12345" }
}
```

8.5.2. Read

Example user read response:

```
{
    "@type": "user",
    "id": "UID-5001",
    "details": {
        "@type": "Card",
        "version": "2.0",
        "kind": "individual",
        "name": { "full": "John Doe" },
        "emails": {
            "email": { "address": "john.doe@example.example" }
        }
    },
    "status": { "@type": "userStatus", "label": "active" },
    "organisation": { "@type": "organisation", "id": "ORG-12345" }
}
```

8.5.3. Update

Example user update request (suspend the user):

```
{
    "@type": "user",
    "status": { "@type": "userStatus", "label": "suspended" },
    "details": {
        "@type": "Card",
        "version": "2.0",
        "kind": "individual",
        "name": { "full": "John Doe" },
        "emails": {
            "email": { "address": "john.doe@example.example" }
        }
    }
}
```

8.5.4. Delete

The user delete operation takes the user identifier as the resource identifier in the context of the owning organisation. No request body is required.

8.5.5. Reference

Example user reference (used when referencing a user from an organisation object):

```
{
    "@type": "user",
    "id": "UID-5001"
}
```

8.6. Message

8.6.1. Create

Example message create request using a Transfer Request Message Object, notifying the recipient organisation that a transfer of the domain name `transfer-test.example` is pending and requires action:

```
{
    "@type": "message",
    "data": {
        "@type": "transferRequestMessage",
        "objectType": "domainName",
        "objectId": "transfer-test.example",
        "process": {
            "@type": "domainTransferProcess",
            "processId": "TRF-1001",
            "trStatus": "pending",
            "transferDir": "pull",
            "reqClientId": "ClientX",
            "requestDate": "2026-09-17T09:00:00.0Z",
            "actClientId": "ClientY",
            "actionDate": "2026-09-22T09:00:00.0Z"
        },
        "status": "pending",
        "requestDate": "2026-09-17T09:00:00.0Z",
        "actionDate": "2026-09-22T09:00:00.0Z"
    }
}
```

Example message create response:

```
{
    "@type": "message",
    "id": "MSG-98765",
    "creationDate": "2026-09-17T09:00:00.0Z",
    "status": "queued",
    "data": {
        "@type": "transferRequestMessage",
        "objectType": "domainName",
        "objectId": "transfer-test.example",
        "process": {
            "@type": "domainTransferProcess",
            "processId": "TRF-1001",
            "trStatus": "pending",
            "transferDir": "pull",
            "reqClientId": "ClientX",
            "requestDate": "2026-09-17T09:00:00.0Z",
            "actClientId": "ClientY",
            "actionDate": "2026-09-22T09:00:00.0Z"
        },
        "status": "pending",
        "requestDate": "2026-09-17T09:00:00.0Z",
        "actionDate": "2026-09-22T09:00:00.0Z"
    }
}
```

8.6.2. Read

Example message read response:

```
{
    "@type": "message",
    "id": "MSG-98765",
    "creationDate": "2026-09-17T09:00:00.0Z",
    "status": "delivered",
    "data": {
        "@type": "transferRequestMessage",
        "objectType": "domainName",
        "objectId": "transfer-test.example",
        "process": {
            "@type": "domainTransferProcess",
            "processId": "TRF-1001",
            "trStatus": "pending",
            "transferDir": "pull",
            "reqClientId": "ClientX",
            "requestDate": "2026-09-17T09:00:00.0Z",
            "actClientId": "ClientY",
            "actionDate": "2026-09-22T09:00:00.0Z"
        },
        "status": "pending",
        "requestDate": "2026-09-17T09:00:00.0Z",
        "actionDate": "2026-09-22T09:00:00.0Z"
    }
}
```

Example message read response with a Review Response Object:

```
{
    "@type": "message",
    "id": "MSG-98770",
    "creationDate": "2026-09-19T14:30:00.0Z",
    "status": "queued",
    "data": {
        "@type": "reviewResponseMessage",
        "objectType": "domainName",
        "objectId": "review-test.example",
        "approved": true,
        "clientTransactionId": "ABC-12345",
        "serverTransactionId": "54322-XYZ",
        "completionDate": "2026-09-19T14:30:00.0Z",
        "message": "The domain name registration was approved."
    }
}
```

Example message read response with a Transfer Outcome Message Object:

```
{
    "@type": "message",
    "id": "MSG-98771",
    "creationDate": "2026-09-22T09:00:00.0Z",
    "status": "queued",
    "data": {
        "@type": "transferOutcomeMessage",
        "objectType": "domainName",
        "objectId": "transfer-test.example",
        "process": {
            "@type": "domainTransferProcess",
            "processId": "TRF-1001",
            "trStatus": "serverApproved",
            "transferDir": "pull",
            "reqClientId": "ClientX",
            "requestDate": "2026-09-17T09:00:00.0Z",
            "actClientId": "ClientY",
            "actionDate": "2026-09-22T09:00:00.0Z"
        },
        "trStatus": "serverApproved",
        "requestDate": "2026-09-17T09:00:00.0Z",
        "actionDate": "2026-09-22T09:00:00.0Z",
        "expiryDate": "2028-04-03T22:00:00.0Z"
    }
}
```

Example message read response with an Auto-Renewal Message Object:

```
{
    "@type": "message",
    "id": "MSG-98772",
    "creationDate": "2026-09-23T00:05:00.0Z",
    "status": "queued",
    "data": {
        "@type": "autoRenewalMessage",
        "objectType": "domainName",
        "objectId": "renewed.example",
        "renewalDate": "2026-09-23T00:05:00.0Z",
        "expiryDate": "2027-09-23T00:00:00.0Z"
    }
}
```

Example message read response with an Expiration Deletion Message Object:

```
{
    "@type": "message",
    "id": "MSG-98774",
    "creationDate": "2026-09-25T00:05:00.0Z",
    "status": "queued",
    "data": {
        "@type": "expirationDeletionMessage",
        "objectType": "domainName",
        "objectId": "expired.example",
        "deletionDate": "2026-09-25T00:00:00.0Z"
    }
}
```

Example message read response with a Service Notice Message Object, which is not linked to a specific object:

```
{
    "@type": "message",
    "id": "MSG-98773",
    "creationDate": "2026-09-24T08:00:00.0Z",
    "status": "queued",
    "data": {
        "@type": "serviceNoticeMessage",
        "message": "The registry terms and conditions have been updated."
    }
}
```

8.6.3. Query

There is no request message for the message query operation.

Example message query response, containing a Transfer Request Message Object for the domain `transfer-test.example`:

```
[{
    "@type": "message",
    "id": "MSG-98765",
    "creationDate": "2026-09-17T09:00:00.0Z",
    "status": "queued",
    "data": {
        "@type": "transferRequestMessage",
        "objectType": "domainName",
        "objectId": "transfer-test.example",
        "process": {
            "@type": "domainTransferProcess",
            "processId": "TRF-1001",
            "trStatus": "pending",
            "transferDir": "pull",
            "reqClientId": "ClientX",
            "requestDate": "2026-09-17T09:00:00.0Z",
            "actClientId": "ClientY",
            "actionDate": "2026-09-22T09:00:00.0Z"
        },
        "status": "pending",
        "requestDate": "2026-09-17T09:00:00.0Z",
        "actionDate": "2026-09-22T09:00:00.0Z"
    }
}]
```

8.7. Discovery

8.7.1. Read

There is no request message for the discovery read operation.

Example discovery read response:

```
{
    "@type": "discovery",
    "versions": [
        {
            "@type": "discoveryVersion",
            "version": "1.0",
            "baseUrl": "https://rpp.example/rpp"
        }
    ],
    "environment": {
        "@type": "discoveryEnvironment",
        "name": "rpp.example",
        "status": "production"
    },
    "tlds": ["example"],
    "extensions": [
        {
            "@type": "extension",
            "name": "RPP example extension",
            "id": "urn:ietf:params:rpp:extension:example-extension",
            "version": "1.0",
            "url": "https://www.iana.org/assignments/rpp-extensions/rpp-
example-extension"
        }
    ],
    "profiles": ["urn:ietf:params:rpp:profile:epp-compatibility"],
    "objects": ["domainName", "host", "contact"],
    "datetime": "2026-05-31T12:30:00Z",
    "links": [
        { "label": "privacy-policy", "object": "https://registry.example/
privacy" },
        { "label": "terms-of-service", "object": "https://registry.example/
terms" }
    ],
    "idnLgr": [
        {
            "@type": "discoveryIdnLgr",
            "tld": ["example"],
            "name": "example-latn-1.0",
            "url": "https://www.iana.org/domains/idn-tables/tables/
example_latn_1.0.txt",
            "default": true
        }
    ],
    "notices": [
        {
            "@type": "discoveryNotice",
            "pubTime": "2026-06-01T00:00:00Z",
            "text": "This server will undergo planned maintenance on the
first Monday of each month."
        }
    ]
}
```

8.8. Profile

8.8.1. Read

There is no request message for the profile read operation.

Example profile read response:

```
{
    "@type": "profile",
    "name": "urn:ietf:params:rpp:profile:epp-compatibility",
    "description": "EPP compatibility profile",
    "version": "1.0",
    "rppVersion": "1.0",
    "objects": ["domainName", "host", "contact"],
    "extensions": [
        {
            "@type": "extension",
            "name": "RPP example extension",
            "id": "urn:ietf:params:rpp:extension:example-extension",
            "version": "1.0",
            "url": "https://www.iana.org/assignments/rpp-extensions/rpp-
example-extension"
        }
    ],
    "defaultProfile": true
}
```

Example profile read response for a profile that extends a base profile:

```
{
    "@type": "profile",
    "name": "org.example.rpp.profile.registry-extended",
    "description": "Registry specific profile extending the EPP compatibility
profile",
    "version": "1.0",
    "rppVersion": "1.0",
    "objects": ["domainName", "host", "contact"],
    "baseProfile": {
        "@type": "profile",
        "name": "urn:ietf:params:rpp:profile:epp-compatibility"
    },
    "defaultProfile": false
}
```

9. IANA Considerations

9.1. RPP Media Type (application/rpp+json)

The IANA is requested to add the "application/rpp+json" media type to the "Media Types" registry, following the template in [RFC6838]

```
Type name: application
Subtype name: rpp+json
Required parameters: "N/A"
Optional parameters: profile, version
Encoding considerations: "N/A"
Security considerations: This type has all of the security
               considerations described in [@!RFC8259] plus the
               considerations specified in the Security Considerations
               section of this document.
               The usage scenarios of the RPP media type do not foresee any
               code execution originating from the content of this media
type.
Interoperability considerations: "N/A"
Published specification: This document
Applications that use this media type: RPP protocol and extensions
Fragment identifier considerations: "N/A"
Additional information:
   Deprecated alias names for this type: "N/A"
   Magic number(s): "N/A"
   File extension(s): "N/A"
   Macintosh file type code(s): "N/A"
Person & email address to contact for further information: Author's email
address
Intended usage: COMMON
Restrictions on usage: "N/A"
Author: Document authors
Change controller: Document authors
Provisional registration: No
```

9.2. RPP JSON Schema Extensions Registry

The IANA is requested to create a new registry for RPP JSON Schema extensions, this registry will be used to register standardised JSON Schema extensions to the JSON representation of RPP JSON objects, as defined in the Section 7 of this document.

```
Name of the registry: RPP JSON Schema Extensions
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Expert Review
```

Fields to be registered:

- `name`: The name of the extension, for example "RPP example JSON extension".
- `id`: The unique `$id` URI of the extension JSON Schema, for example "https://www.iana.org/assignments/rpp-json-extensions/example-extension.json".
- `version`: The version of the extension, for example "1.0".
- `RFC`: The RFC number for the extension specification, for example "RFC 1234".
- `description`: A human-readable description of the extension and its intended use.

The "RESTful Provisioning Protocol (RPP)" registry group is defined in [I-D.ietf-rpp-core].

TODO there already exists a registry for RPP extensions, defined in the rpp-core document, consider whether this new registry is necessary or if it should be merged with the existing one.

10. Internationalization Considerations

Internationalized Domain Names (IDN) are represented using the paired `name`/`uName` data elements described in the Domain Name Data Object. The `name` property, which addresses the resource and is used for object identity, MUST always contain the ACE A-label form; the `uName` property carries the Unicode U-label form for display and MUST be Unicode Normalization Form C (NFC) as defined in [UNICODE.NFC].

Because JSON strings are Unicode character sequences, implementations MUST take the following precautions to avoid producing or misinterpreting `uName` (and any other Unicode-bearing string value) in a way that breaks common JSON consumers, in particular JavaScript, whose native strings are sequences of UTF-16 code units rather than Unicode code points:

- A string value MUST NOT contain an unpaired (lone) UTF-16 surrogate code point. Although [RFC8259], Section 8.2, permits such values in JSON text, they do not represent a well-formed Unicode scalar value and MAY fail to round-trip when re-encoded as UTF-8 by non-JavaScript consumers.
- Consumers MUST NOT use a UTF-16 code-unit count (such as JavaScript's `String.prototype.length`) to validate or enforce length constraints on `uName`. Domain name label length restrictions are defined in terms of the `name` (A-label) octet count; scripts using supplementary-plane code points are represented as surrogate pairs in UTF-16, so a code-unit count over-counts the number of Unicode code points.
- Consumers MUST NOT rely on native string equality (e.g. JavaScript's `===`) to compare `uName` values for identity purposes, since two strings that differ only in Unicode normalisation form compare unequal even though they represent the same name; `name` MUST be used instead for identity comparison.
- Servers MUST emit `name` in lowercase. Because ACE (A-label) domain names are case-insensitive per DNS comparison rules but JSON/JavaScript string comparison is case-sensitive, clients performing equality checks on `name` MUST lowercase both operands first rather than relying on native string equality.

11. Security Considerations

If a server accepts a `uName` value without strictly verifying both that it is already Unicode Normalization Form C (NFC) and that it converts, via the ToASCII procedure of Section 4.2, to the exact supplied `name` value, an attacker could cause a displayed `uName` to diverge from the name actually resolved in the DNS. Servers MUST reject any create or update request where `uName` is not already NFC-normalised or does not convert to the supplied `name`; servers MUST NOT silently normalise or correct the value on the client's behalf.

TODO

12. Acknowledgments

TODO

13. Change History

13.1. Version draft-ietf-rpp-json-01 to draft-ietf-rpp-json-02

13.2. Version draft-ietf-rpp-json-00 to draft-ietf-rpp-json-01

- Added section "JSON Schema Composition Rules", including Section 4.10.1 and `read`/`create` views for Component Objects. (Issue #102)
- Replaced the JSON Schema definitions the generic Status Object with type-specific objects in Section 6.2.3.
- Added schemas and examples for the Availability Objects, service messages, and for the Discovery and Profile Data Objects. (Issues #133, #82, #92)
- Added IANA registration for the "application/rpp+json" media type. (Issue #84)
- Added Section 7. (Issue #78)
- Added support for Internationalized Domain Names (IDN), see Section 10. (Issue #86)
- Added section "Inheritance". (Issue #88)
- Added section describing the representation of undefined values. (Issue #97)
- Rewrote the partial update processing in Section 5.2.2, added replacement of a whole array, and added flow figures. (Issue #91, #77)
- Aligned the JSContact constraints with the RPP JSContact profile. (Issue #76)
- Made the JSON Schema definitions normative. (Issue #75)
- Added the Disclose Object and Process Object (Base, Create, Restore) schemas, and removed redundant Process alias and Transfer response schemas.
- Aligned all examples, including those in Section 7, with the updated JSON Schema definitions.
- Generalised the Mutability Rules to derive request and response content from operation Input and Output, and changed handling of disallowed request data elements from ignore to reject. (Issue #112 M2)
- Replaced the primary identifier with the Unique Identifier in Association Rules.
- Editorial: Removed rule numbering and replaced references to rule numbers with section references.
- Editorial: Updated the Abstract and Introduction to cover all defined Data Objects.

13.3. Version draft-wullink-rpp-json-02 to draft-ietf-rpp-json-00

- Added "Update Rules" section, describing update requests (Issue #54)
- Added schema and examples for Transfer approve/reject/cancel operations (Issue #28)
- Added Organisation and User Object JSON schemas and examples. (Issue #57)
- Added schema and examples for the Renew Process Object. (Issue #45)
- Added representation for embedding of process data in uniform interface operations (especially create). (Issue #60)

- Added JSContact as External Type Embedding (Issue #43).

13.4. Version 01 to 02

- Adjust rule 19 and all schemas to match it.

13.5. Version 00 to 01

- Updated all examples and schemas to be based on RPP Data Object and no longer on EPP XML schemas. (Issue #15)
- Updated labelled and dictionary aggregation rules (Issue #17)
- Added required "@type" property to all JSON Schema definitions. (Issue #20)
- Updated all example domain names to use the .example TLD. (Issue #26)

14. References

14.1. Normative References

[I-D.ietf-jsonschema-json-schema] Dusseault, L. M., Wright, A., and H. H. Andrews, "JSON Schema", Work in Progress, Internet-Draft, draft-ietf-jsonschema-json-schema-04, 13 October 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-jsonschema-json-schema-04>.

[I-D.ietf-rpp-core] Wullink, M. and P. Kowalik, "RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-ietf-rpp-core-00, 6 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-rpp-core-00>.

[IDN-Tables] Internet Assigned Numbers Authority (IANA), "Repository of IDN Practices", <https://www.iana.org/assignments/idn-tables>.

[ISO3166-1] International Organization for Standardization, "Codes for the representation of names of countries and their subdivisions - Part 1: Country code", ISO 3166-1:2020, 2020, <https://www.iso.org/standard/72482.html>.

[ITU.E164.2005] International Telecommunication Union, "The international public telecommunication numbering plan", ITU-T Recommendation E.164, February 2005.

[RFC1035] Mockapetris, P., "Domain names - implementation and specification", STD 13, RFC 1035, DOI 10.17487/RFC1035, November 1987, <https://www.rfc-editor.org/info/rfc1035>.

[RFC1738] Berners-Lee, T., Masinter, L., and M. McCahill, "Uniform Resource Locators (URL)", RFC 1738, DOI 10.17487/RFC1738, December 1994, <https://www.rfc-editor.org/info/rfc1738>.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>.

[RFC3339] Klyne, G. and C. Newman, "Date and Time on the Internet: Timestamps", RFC 3339, DOI 10.17487/RFC3339, July 2002, <https://www.rfc-editor.org/info/rfc3339>.

[RFC3915] Hollenbeck, S., "Domain Registry Grace Period Mapping for the Extensible Provisioning Protocol (EPP)", RFC 3915, DOI 10.17487/RFC3915, September 2004, <https://www.rfc-editor.org/rfc/rfc3915>.

[RFC4648] Josefsson, S., "The Base16, Base32, and Base64 Data Encodings", RFC 4648, DOI 10.17487/RFC4648, October 2006, <https://www.rfc-editor.org/info/rfc4648>.

[RFC5730] Hollenbeck, S., "Extensible Provisioning Protocol (EPP)", STD 69, RFC 5730, DOI 10.17487/RFC5730, August 2009, <https://www.rfc-editor.org/info/rfc5730>.

[RFC5732] Hollenbeck, S., "Extensible Provisioning Protocol (EPP) Host Mapping", STD 69, RFC 5732, DOI 10.17487/RFC5732, August 2009, <https://www.rfc-editor.org/info/rfc5732>.

[RFC5890] Klensin, J., "Internationalized Domain Names for Applications (IDNA): Definitions and Document Framework", RFC 5890, DOI 10.17487/RFC5890, August 2010, <https://www.rfc-editor.org/info/rfc5890>.

[RFC5891] Klensin, J., "Internationalized Domain Names in Applications (IDNA): Protocol", RFC 5891, DOI 10.17487/RFC5891, August 2010, <https://www.rfc-editor.org/info/rfc5891>.

[RFC6838] Freed, N., Klensin, J., and T. Hansen, "Media Type Specifications and Registration Procedures", BCP 13, RFC 6838, DOI 10.17487/RFC6838, January 2013, <https://www.rfc-editor.org/info/rfc6838>.

[RFC6902] Bryan, P., Ed. and M. Nottingham, Ed., "JavaScript Object Notation (JSON) Patch", RFC 6902, DOI 10.17487/RFC6902, April 2013, <https://www.rfc-editor.org/info/rfc6902>.

[RFC8259] Bray, T., Ed., "The JavaScript Object Notation (JSON) Data Interchange Format", STD 90, RFC 8259, DOI 10.17487/RFC8259, December 2017, <https://www.rfc-editor.org/info/rfc8259>.

[RFC8543] Zhou, L., Kong, N., Yao, J., Gould, J., and G. Zhou, "Extensible Provisioning Protocol (EPP) Organization Mapping", RFC 8543, DOI 10.17487/RFC8543, March 2019, <https://www.rfc-editor.org/info/rfc8543>.

[RFC9553] Stepanek, R. and M. Loffredo, "JSContact: A JSON Representation of Contact Data", RFC 9553, DOI 10.17487/RFC9553, May 2024, <https://www.rfc-editor.org/info/rfc9553>.

[RFC9982] Stepanek, R., "JSContact Version 2.0: A JSON Representation of Contact Data", RFC 9982, DOI 10.17487/RFC9982, May 2026, <https://www.rfc-editor.org/info/rfc9982>.

14.2. Informative References

[I-D.ietf-rpp-data-objects] Kowalik, P. and M. Wullink, "RESTful Provisioning Protocol (RPP) Data Objects", Work in Progress, Internet-Draft, draft-ietf-rpp-data-objects-01, 3 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-rpp-data-objects-01>.

[I-D.simmen-rpp-dns-data] Simmen, C. and P. Kowalik, "DNS data representation for use in RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-simmen-rpp-dns-data-01, 20 October 2025, <https://datatracker.ietf.org/doc/html/draft-simmen-rpp-dns-data-01>.

[RFC3596] Thomson, S., Huitema, C., Ksinant, V., and M. Souissi, "DNS Extensions to Support IP Version 6", STD 88, RFC 3596, DOI 10.17487/RFC3596, October 2003, <https://www.rfc-editor.org/info/rfc3596>.

[RFC4034] Arends, R., Austein, R., Larson, M., Massey, D., and S. Rose, "Resource Records for the DNS Security Extensions", RFC 4034, DOI 10.17487/RFC4034, March 2005, <https://www.rfc-editor.org/info/rfc4034>.

[RFC5910] Gould, J. and S. Hollenbeck, "Domain Name System (DNS) Security Extensions Mapping for the Extensible Provisioning Protocol (EPP)", RFC 5910, DOI 10.17487/RFC5910, May 2010, <https://www.rfc-editor.org/info/rfc5910>.

[UNICODE.NFC] Unicode Consortium, "Unicode Normalization Forms", August 2026, <https://www.unicode.org/reports/tr15/>.

Authors' Addresses

Maarten Wullink
SIDN Labs
Email: maarten.wullink@sidn.nl
URI: https://sidn.nl/

Pawel Kowalik
DENIC
Email: pawel.kowalik@denic.de
URI: https://denic.de/